



University of Applied Sciences and Arts Northwestern Switzerland FHNW

Master of Science in Engineering

Energie-Management in mobilen Robotern

Author:	Andri Wild
Advisor:	Prof. Dr. Christoph Stamm
Start Date:	17.03.2025
Submission Date:	31.08.2025

Abstract

Diese Arbeit erweitert ein Batterie-Management-System für mobile Roboter, basierend auf Extended Kalman Filtern zur Schätzung von State of Charge (SOC) und State of Health (SOH) von AGM-Blei-Säure-Batterien. Das System wurde für den Roboter des Projekts *Werteorientierter Schweizer Roboter* entwickelt, der zur Unterstützung des Personals in Pflegeheimen eingesetzt wird.

Die Batterie-Charakterisierung erfolgte durch einen Hybrid Pulse Power Characterization Test, aus dem ein Equivalent Circuit Model abgeleitet wurde. Der SOC-Extended Kalman Filter nutzt dieses Modell zur Ladezustandsbestimmung, während ein separater SOH-Filter die verfügbare Kapazität schätzt. Die Implementation wurde in das bestehende Robot Operating System (ROS) integriert.

Validierungsversuche zeigen deutliche Verbesserungen gegenüber bisherigen spannungsbasierten SOC-Schätzungen. Das System erreicht stabile Resultate bei unterschiedlichen Belastungen und ermöglicht Reichweitenberechnungen auf Basis der verfügbaren Batteriekapazität.

Das entwickelte Battery Management System bildet die Grundlage für Ladestrategien und prädiktive Wartung. Auf diesen Informationen lassen sich intelligente Erweiterungen umsetzen, die zu einem effizienten und zuverlässigen Betrieb beitragen.

Inhaltsverzeichnis

1	Einleitung	5
1.1	Ausgangslage	5
1.2	Zielsetzung	5
2	Grundlagen	6
2.1	Systembeschreibung	7
2.2	Batterie	8
2.2.1	AGM Blei-Säure Batterien	8
2.2.2	Eigenschaften	8
2.2.3	Batterie Betrieb	9
2.2.4	Alterung	10
2.3	Battery Management System	11
2.3.1	State of Charge	11
2.3.2	State of Health	14
2.3.3	Kalman Filter	14
2.3.4	Extended Kalman Filter	24
2.3.5	Zusammenfassung	27
2.4	Robot Operating System (ROS)	28
2.4.1	Architektur	28
2.4.2	Nachrichten und Datentypen	28
3	Umsetzung	29
3.1	SOC - Extended Kalman Filter	30
3.1.1	Systemmodell	30
3.1.2	Implementierung EKF	36
3.2	SOH - Extended Kalman Filter	41
3.2.1	Systemmodell	41
3.2.2	Implementierung EKF	41
3.3	Roboterintegration	43
3.3.1	Datenerfassung	43
3.3.2	ROS Integration	43
3.4	Energieverbrauch	46
3.4.1	Analyse Stromverbrauch	46
3.4.2	Reichweitenabschätzung	48
3.4.3	Ladestrategie	48
3.5	Monitoring	51
4	Validierung	52
4.1	State of Charge	52

4.1.1 Funktionstest	52
4.1.2 Praktische Anwendung	53
4.1.3 Spannungsbasierte SOC-Schätzung	60
4.1.4 Abweichung bei der Ladung	60
4.2 State of Health	60
4.3 Temperatur	61
5 Fazit	62
6 Ausblick	63
Abbildungsverzeichnis	64
Bibliographie	66

1 Einleitung

Elektrische Energie zu speichern ist auch heute noch eine grosse Herausforderung. Noch nie zuvor waren so viele Fahrzeuge von Elektromotoren angetrieben, wodurch das Forschungsfeld der Akkumulatoren weiter an Bedeutung gewinnt. Nebst vielen vorteilhaften Eigenschaften von Akkumulatoren sind genaue Messungen in Bezug auf die verfügbare Energiemenge und den Zustand schwierig. Dies ist der Bauweise und den physikalischen Eigenschaften sowie den vielen Faktoren geschuldet, die den Energiespeicher beeinflussen können. Um möglichst präzise Abschätzungen des Ist-Zustands zu machen, existieren diverse Ansätze. Einige davon verwenden direkte Messungen von physikalischen Eigenschaften, andere basieren auf mathematischen Modellen. Gerade die Industrie der elektrischen Fahrzeuge ist auf ein akkurates Energiemanagement angewiesen, wodurch das Forschungsfeld weiter an Bedeutung gewann.

1.1 Ausgangslage

Im Projekt *Werteorientierter Schweizer Roboter* (WSR) wird erforscht, wie ein autonomer Roboter in einem Heim für Pflegebedürftige nützlich und auf empathische sowie soziale Weise eingesetzt werden kann. Die vorgesehenen Aufgaben umfassen in erster Linie das Aufsuchen von Heimbewohnern sowie das Begleiten zu Therapiesitzungen. Dies soll das Pflegepersonal entlasten und Freiräume für die Pflege schaffen. Um dies zu ermöglichen, ist der Roboter mit modernster Technik ausgestattet. Diese erlaubt es ihm, sich auf einer Karte zu lokalisieren und autonom an einem Ziel zu navigieren. Akkumulatoren stellen dabei die Stromversorgung dar, welche dadurch ein essenzielles Element des Gesamtsystems darstellt. Nur wenn der Roboter genügend Energie zur Verfügung hat, kann dieser seine Aufgaben zuverlässig erledigen. Entladene Akkumulatoren lädt der Roboter typischerweise an einem Dock, eine Lade-Einrichtung, die er eigenständig aufsuchen kann. Insbesondere für den nachhaltigen Dauerbetrieb bedarf es eines entsprechenden Energiemanagements.

1.2 Zielsetzung

In dieser Arbeit wird untersucht, wie ein Energiemanagement für den im Projekt WSR eingesetzten Roboter umgesetzt werden kann. Dabei stehen die folgenden Forschungsfragen im Vordergrund:

- Wie gross ist der Energieverbrauch in praktischen Anwendungsszenarien?
- Welche Modelle eignen sich, um realistische Reichweitenabschätzungen bei unterschiedlichen Verbrauchsmustern und unterschiedlichem Batteriealter zu ermöglichen?
- Wie sieht eine sinnvolle Ladestrategie aus, um den unterschiedlichen Ansprüchen (Batterielebensdauer, Roboter-Verfügbarkeit) gerecht zu werden?

2 Grundlagen

Die Überwachung und Steuerung von Energiespeichern ist entscheidend für Effizienz, Sicherheit und Zuverlässigkeit von elektrischen Systemen im Betrieb. Dazu behandelt dieses Kapitel die theoretischen Grundlagen zur Entwicklung eines Überwachungssystems von Batterien.

Nach einer kurzen Systembeschreibung werden die relevanten Eigenschaften von Batterien aufgeführt. Dies schliesst Charakteristiken, Verhalten unter verschiedenen Belastungsbedingungen und Alterungsmechanismen ein. Diese Grundlagen sind die Basis für das Verständnis der Herausforderungen, die sich bei der Messung von Batterie-Parameter stellen. Anschliessend werden Messsysteme und Schätzverfahren für Batteriekenngrössen wie *State of Charge* (SOC) und *State of Health* (SOH) untersucht. Dabei wird die Auswahl des Extended Kalman Filters als gute Methode für das nichtlineare Verhalten von Batteriesystemen begründet und dessen theoretische Grundlagen werden erläutert.

Den Abschluss bildet eine kurze Einführung in das Robot Operating System (ROS). ROS wird häufig in Robotik-Systemen eingesetzt und bietet eine modulare sowie skalierbare Systemarchitektur für komplexe Anwendungen.

2.1 Systembeschreibung

Die Umsetzung des Roboters für den Pflegeheimeinsatz ist noch nicht abgeschlossen. Für die Entwicklung des Energiemanagement-Systems wird daher eine technisch vergleichbare Roboterplattform verwendet. Diese verfügt über identische Funktionalitäten, um das Projekt auf verschiedenen Ebenen voranzutreiben. Die Plattform wurde von der kanadischen Firma Clearpath für Ausbildungs- und Forschungszwecke entwickelt [1]. Durch den Antrieb über Omniwheels handelt es sich um ein holonomisches System. Für die Navigation und Lokalisierung sind zwei Lidars und eine IMU verbaut. Der Roboter wird über einen zentralen Computer gesteuert, der sich im Gehäuse des Roboters befindet. Die folgende Abbildung 1 stellt ein 3D-Modell des Roboters dar.



Abbildung 1: 3D-Modell WSR-Roboter

Um die Technologien für den Einsatz im Wohnheim zu testen, wurde der Roboter mit einem Aufbau und zusätzlichen Komponenten ausgestattet. Auf der Grundplatte ist ein Jetson-Computer installiert, der vor allem die Personen- und Objekterkennung auf Video-Streams übernimmt. Mehrere Sonar-Sensoren umgeben den Jetson und detektieren transparente Hindernisse wie Fenster, die für das Lidar unsichtbar sind. Vier Kameras in erhöhter Position bieten eine vollständige Rundumsicht der Umgebung. Die Kamerastreams werden über zwei Raspberry-Pi-Computer an den Jetson weitergeleitet. Für die Interaktion mit Bewohnern und Personal verfügt der Roboter über einen Lautsprecher mit integriertem Mikrofon. Zudem ist ein kleines Touchdisplay für einfache Eingaben oder Statusanzeigen vorhanden. Das gesamte System wird über zwei 12 V Batterien mit je 22 Ah versorgt, die sich im Gehäuse der Roboterplattform befinden.

2.2 Batterie

Bei Batterien unterscheidet man grundsätzlich zwischen Primär- und Sekundärzellen. Primärzellen entladen sich nur einmal, während Sekundärzellen (Akkumulatoren) mehrfache Verwendung ermöglichen. Am weitesten verbreitet unter den Sekundärzellen sind Lithium-Ionen-, Nickel-Metallhydrid- und Blei-Säure-Akkus. Sie unterscheiden sich insbesondere in Bezug auf Energiedichte, Lebensdauer, Ladeverhalten, Selbstentladung und Kosten. In dieser Arbeit kommen als Energiespeicher zwei versiegelte Blei-Säure-Batterien der AGM-Bauart zum Einsatz. Aufgrund seiner Bauart ist er wartungsfrei und eignet sich für den Einsatz in Robotik-Systemen.

2.2.1 AGM Blei-Säure Batterien

AGM steht für *Absorbent Glass Mat* und bezeichnet eine Blei-Säure-Technologie, die gegenüber älteren Varianten einige Vorteile bietet. Der wesentliche Unterschied liegt in der Speicherung des Elektrolyts. Während bei konventionellen Batterien flüssiges Elektrolyt verwendet wird, ist dieses bei AGM-Batterien in Glasfasermatten fixiert. Dadurch ist ein wartungsfreier und auslaufsicherer Betrieb möglich. Die verbesserte Zyklenfestigkeit macht AGM-Batterien ideal für Deep-Cycle-Anwendungen in autonomen Robotersystemen. *Deep-Cycle* bedeutet, dass der Akkumulator für viele Zyklen ohne signifikante Leistungseinbußen verwendet werden kann [2]. Die folgende Abbildung 2 zeigt den technischen Aufbau einer AGM-Batterie.

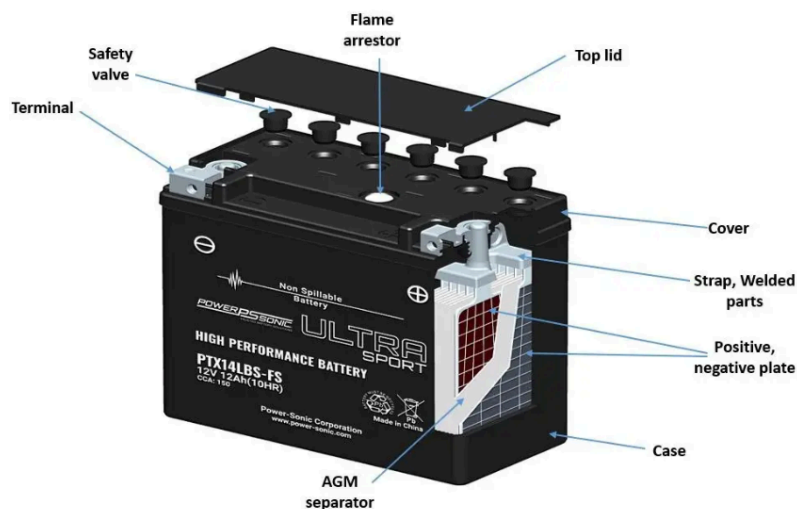


Abbildung 2: Konstruktion einer AGM Blei-Säure Batterie (Quelle: [2])

2.2.2 Eigenschaften

Eine Blei-Säure-Batterie liefert typischerweise eine Zellenspannung von etwa 2 Volt. Ihre Energiedichte liegt bei rund 29–44 Wh/kg. Durch das Schalten mehrerer solcher Zellen in Serie oder parallel lassen sich unterschiedliche Spannungen und Leistungen kombinieren. Die im Roboter eingesetzte Batterie weist eine Nennspannung von 12 Volt auf und hat somit sechs Zellen in Serie geschaltet. Zur Erhöhung der Kapazität wird eine zweite, identische Batterie parallel betrieben [3].

Die folgenden Kenngrößen sind im Allgemeinen für Batteriesysteme relevant:

- Die **Kapazität** bezeichnet die speicherbare Ladung in Ah oder mAh. Berechnet wird die Kapazität durch die Stromstärke mal Zeit. Fließt eine Stunde lang zwei Ampere, so ergibt dies 2 Ah. Die verfügbare Kapazität in Prozent wird auch *State of Charge* (SOC) genannt.
- **C-Rate** ist ein Begriff, um den Lade- oder Entladestrom in Relation zur Kapazität anzugeben. Die C-Rate berechnet sich durch Division von Strom zur Kapazität. Eine hohe C-Rate bedeutet, dass grosse Ströme fließen, d.h., die Batterie wird entweder schnell ge- oder entladen [4].
- Die **Zyklen-Lebensdauer** beschreibt die Anzahl Lade-Entlade-Zyklen bis zu einem definierten Kapazitätsverlust.
- Als **Selbstentladung** bezeichnet man den Kapazitätsverlust bei Lagerung.
- Der **Innenwiderstand** wirkt sich auf Lade- und Entladeleistung aus. Ein niedriger Innenwiderstand ermöglicht höhere Ströme und geringere Verluste. Der Innenwiderstand definiert somit den maximalen Kurzschlussstrom und verändert sich je nach Ladestand und Alter der Batterie.
- Die **Open Circuit Voltage (OCV)** bezeichnet die Leerlaufspannung der Batterie. Die Spannung wird erst nach einer Ruhephase ohne Belastung präzise erreicht.

2.2.3 Batterie Betrieb

Aufgrund komplexer, nichtlinearer Zusammenhänge weisen Batteriesysteme ein schwer vorher-sagbares Verhalten auf. Produktionsvariationen, Temperatur, Alter, Einsatzbedingungen und Lagerung beeinflussen den Alterungsprozess und somit die verfügbare Kapazität. Ein sachgerechter Betrieb ist daher essenziell für eine konstante Batterieleistung. Die folgenden Abschnitte erläutern die wichtigsten Einflussfaktoren im Detail.

2.2.3.1 Tiefe der Entladung

Die Tiefe der Entladung in einem Zyklus wird *Depth of Discharge* (DoD) genannt. Diese gibt an, wie viel Restkapazität vor dem nächsten Ladezyklus vorhanden ist. Der Wert steht in direktem Zusammenhang mit der Anzahl der Zyklen, die eine Batterie betrieben werden kann [5]. Folgende Abbildung 3 zeigt eine Grafik aus dem Datenblatt der Batterie.

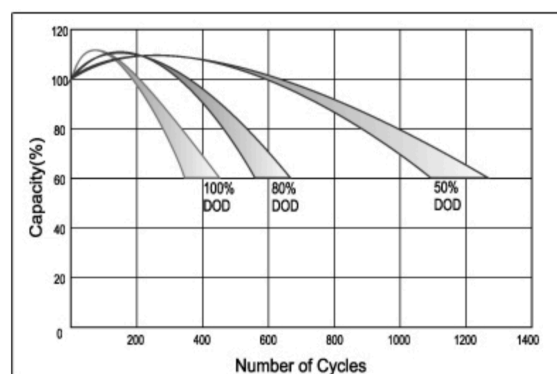


Abbildung 3: Auswirkung des DOD auf die Zyklenzahl (Quelle: [6])

Aus der Grafik ist zu entnehmen, dass bei Halbierung der Entladetiefe die Zyklenzahl auf etwa das dreifache steigt. Komplette Entladungen beschleunigen somit das altern der Batterie.

2.2.3.2 Lagerung

Bei längeren Lagerzeiten entlädt sich die Batterie abhängig von der Umgebungstemperatur. Neben der Selbstentladung ist jedoch die Sulfation problematischer. Durch das Absetzen des Sulfats auf den Elektroden reduziert sich die Kapazität dauerhaft. Daher sollte eine Batterie nach Gebrauch wieder aufgeladen werden [5], [7].

2.2.3.3 Verfügbare Kapazität

Die auf der Batterie und im Datenblatt angegebene Kapazität kann nur unter bestimmten Bedingungen erreicht werden. Dem Datenblatt [6] ist zu entnehmen, dass die verwendete Batterie die angegebenen 22 Ah nur bei einer Belastung von C20 erreicht. Bei höherem Entladestrom ist die verfügbare Kapazität reduziert. Dieses Verhalten ist unter anderem dem Peukert-Effekt geschuldet. Dieser beschreibt das Speichervermögen in Abhängigkeit vom Entladestrom. Der Effekt ist durch zwei physikalische Eigenschaften bedingt. Einerseits ist durch die höhere Belastung der Spannungsabfall über dem Innenwiderstand höher, wodurch es zu einem frühzeitigen Erreichen der Entladeschlussspannung kommt. Der zweite Grund ist die begrenzte Geschwindigkeit der elektrochemischen Prozesse und der Vorgänge des Ladungstransports im Zellinneren der Batterie [8].

2.2.4 Alterung

Die Bestimmung der effektiven Batteriekapazität über die Zeit ist eine zentrale Herausforderung. Der Prozess nennt sich *Capacity-Fade* und beschreibt die kontinuierliche Abnahme der Kapazität über die Zeit. Es sind zwei wesentliche Alterungsmechanismen zu unterscheiden: *Calendar-Aging* bezeichnet den zeitbedingten Kapazitätsverlust, der unabhängig von der Nutzung auftritt, während *Cycling-Aging* durch die Lade-Entlade-Zyklen verursacht wird. Unsachgemässer Betrieb beschleunigt die Alterungsprozesse erheblich [9], [10].

2.2.4.1 Lebensdauer

Der „State of Health“ (SOH) beschreibt den aktuellen Gesundheitszustand und gibt die verfügbare Kapazität im Verhältnis zur Nennkapazität in Prozent an [10]. Typischerweise wird eine Batterie ersetzt, wenn sie noch 80 % ihrer ursprünglichen Kapazität aufweist.

2.3 Battery Management System

In modernen mobilen Robotern kommt für das Energiemanagement häufig ein *Battery Management System* (BMS) zum Einsatz. Die neun Hauptfunktionen eines typischen BMS sind [11]:

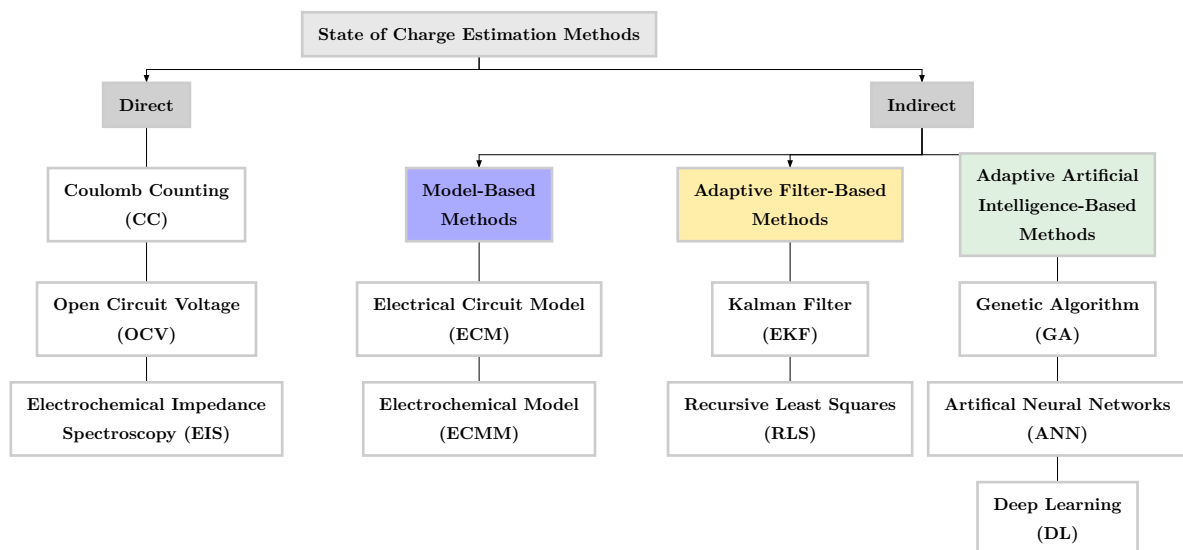
1. Zellbalancierung
2. Thermomanagement
3. Lade- und Entladesteuerung
4. Spannungs- und Stromüberwachung
5. SOC-Schätzung
6. SOH-Schätzung
7. Datenprotokollierung
8. Fehlererkennung und Diagnose
9. Wartung

Die ersten zwei Funktionen sind aufgrund fehlender technischer Voraussetzungen nicht vorhanden. Die Funktionalitäten drei und vier sind von der Roboterplattform bereits umgesetzt. In dieser Arbeit wird das BMS um die Funktionen 5 - 7 erweitert. Im Folgenden werden Methoden aufgezeigt, wie sich SOC- und SOH-Schätzungen für einen mobilen Roboter umsetzen lassen.

2.3.1 State of Charge

Die Bestimmung des SOC ist eine Herausforderung, da dieser Wert nicht direkt messbar ist. Für eine präzise SOC-Bestimmung ist eine vollständige Entladung unter standardisierten Bedingungen erforderlich. Allerdings ist dieser Prozess zeitaufwendig und erfordert spezialisierte Messtechnik. Dies ist für ein BMS keine Lösung, daher wird der SOC in der Praxis mithilfe unterschiedlichen Verfahren geschätzt.

Das folgende Diagramm zeigt eine Auswahl typischer Methoden zur Schätzung des SOC [12].



Bei den direkten Messverfahren wird anhand direkt messbarer physikalischer Eigenschaften auf den SOC geschlossen. Indirekte Methoden schätzen den SOC anhand mathematischer Modelle oder Algorithmen [12]. Nach Recherchen wurden die folgenden Methoden genauer betrachtet.

Beim **Coulomb-Counting (CC)** wird der Lade- und Entladestrom gemessen und über die Zeit integriert. Daraus kann auf die in der Batterie enthaltene Kapazität zurückgeschlossen werden.

Die **Open-Circuit-Voltage (OCV)**-Methode nutzt die Beziehung zwischen offener Klemmenspannung und SOC. Für eine präzise Messung ist eine vorgängige Ruhephase notwendig, so dass sich die Spannung am OCV-Wert angleicht. Voraussetzung für dieses Verfahren ist ein ungefährender linearer Zusammenhang zwischen SOC und OCV.

Der **Kalman-Filter (KF)** basiert auf einem mathematischen Modell und effektiven Messwerten. Durch clevere Kombination lässt sich der SOC während des Betriebs schätzen.

2.3.1.1 Bewertung SOC-Methode

Da die SOC-Schätzung während des laufenden Betriebs erfolgen muss, sind ausschliesslich Online-Messmethoden geeignet. Offline-Verfahren, die eine Unterbrechung des Betriebs oder eine vollständige Entladung der Batterie erfordern würden, scheiden aus praktischen Gründen aus.

Im Rahmen dieser Arbeit werden Machine-Learning-basierte Ansätze zur Kapazitätsbestimmung nicht verfolgt. Obwohl solche Verfahren in der aktuellen Forschung zunehmend an Bedeutung gewinnen, bringen sie einen erheblichen Komplexitätszuwachs mit sich, der in dieser Arbeit nicht in Betracht gezogen wird.

Unter den etablierten Online-Verfahren haben sich das Coulomb-Counting und Kalman-Filter-basierte Methoden als bewährte Lösungen herausgestellt. Beide Ansätze verfügen über eine gute Dokumentation in der Fachliteratur und sind in der Praxis vielfach erprobt.

Der Kalman-Filter-Ansatz bietet gegenüber dem Coulomb Counting mehrere entscheidende Vorteile. Coulomb Counting ist anfällig für die Akkumulation von Messfehlern und bietet keine Möglichkeit zur Selbstkorrektur. Der Kalman-Filter dagegen nähert sich durch clevere Kombination von Schätzung und Messsignalen dem wahren Wert kontinuierlich an. Auch bei abweichender Initialisierung konvergiert der Filter nach einer gewissen Zeit zu einem akkuraten Resultat, wodurch eine robustere Schätzung entsteht.

Der folgende Versuchsaufbau zeigt die Unterschiede der beiden Messverfahren in Matlab Simulink [13]. Auf der Abbildung 4 ist in blau der elektrische Stromkreis mit einer Batterie ersichtlich. Die Strom- und Spannungssignale werden jeweils mit Noise angereichert, um realitätsgetreue Daten zu generieren.

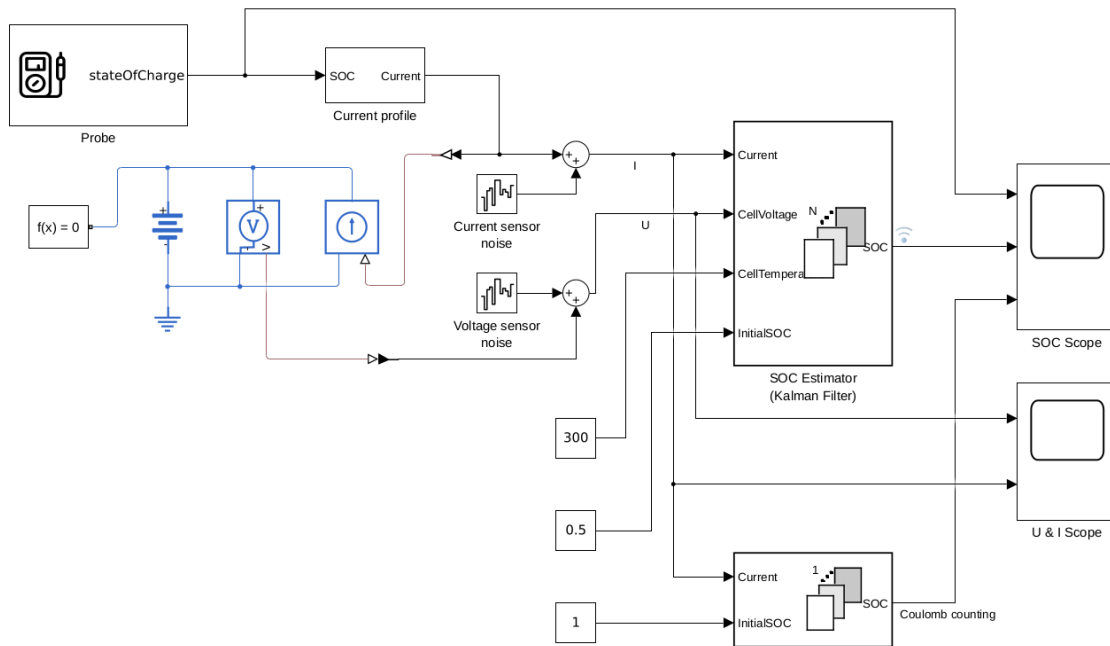


Abbildung 4: Simulink Aufbau Kalman Filter vs. CC

Die *Probe* greift den tatsächlichen SOC-Wert der Batterie ab, welcher als Referenz für die Messsignale dient. Zudem wird mittels *Current profile* die Richtung des Stromflusses gesteuert, um die entladene Batterie wieder zu laden. Die folgende Abbildung 5 zeigt in blau den Spannungsverlauf und in rot den Stromverlauf im Schaltkreis. Nach der Entladephase bei run 10 A folgt eine Ladephase mit 15 A.

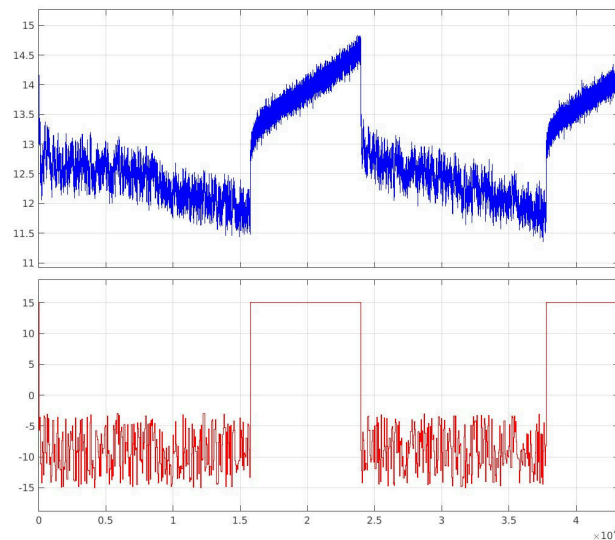


Abbildung 5: Simulink Strom- und Spannungsverlauf

Die Abbildung 6 visualisiert die Resultate der beiden SOC-Estimator.

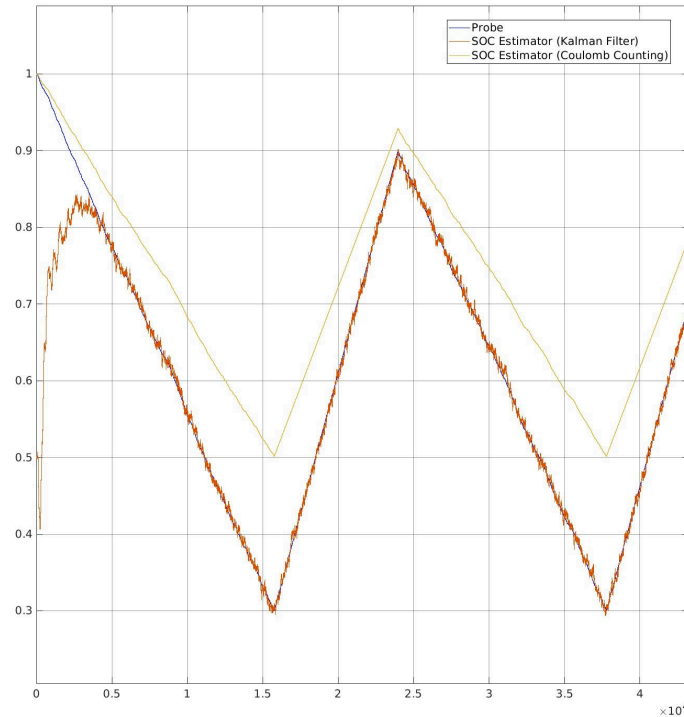


Abbildung 6: Simulink Aufbau Kalman Filter vs. CC

Während in Gelb der Coulomb-Counting Estimator nach kurzer Zeit trotz korrekter Initialisierung eine beachtliche Differenz zum wahren SOC-Wert aufweist, ist der Kalman Filter mit etwas Rauschen jeweils sehr nahe am wahren Wert. Bemerkenswert dabei ist die Korrektur zu Beginn bei einer falschen initialen SOC-Annahme.

Coulomb-Counting lässt sich sehr einfach umsetzen, während der Kalman-Filter deutlich höhere Komplexität mit sich bringt und mehr Rechenaufwand erfordert. Dennoch überzeugt der Kalman-Filter durch seine überlegene Performance. Der Coulomb-Counting-Ansatz bringt schwer lösbare Herausforderungen mit sich, wodurch der zusätzliche Entwicklungsaufwand für einen Kalman-Filter gerechtfertigt ist.

2.3.2 State of Health

Um den Gesundheitszustand der Batterie zu schätzen, gibt es ebenfalls verschiedene Methoden [11]. Da für den SOC ein Kalman Filter verwendet wird, bietet es sich an, dieselbe Methode zu verwenden. Dies ist keine ungewöhnliche Lösung und wird in der Literatur als Dual Extended Kalman Filter (DEKF) betitelt [10].

2.3.3 Kalman Filter

Der Kalman-Filter wurde 1960 von Rudolf Kalman veröffentlicht und basiert auf dem bereits bekannten Prinzip der Alpha-Beta-Filter. Alpha-Beta-Filter bilden eine Filterfamilie, die hauptsächlich zur Glättung von Zeitreihendaten eingesetzt wird. Da diese Filter die theoretische Grundlage für den Kalman-Filter bilden, werden sie im Folgenden Abschnitt näher betrachtet.

2.3.3.1 Alpha Beta Filter

Die Filter arbeiten jeweils mit diskreten Daten, da diese grundsätzlich in Systemen Verwendung finden, in denen Sensordaten über die Zeit verarbeitet werden. Sensordaten sind immer mit einem gewissen Fehler zu betrachten, da Messungen jeweils nur eine begrenzte Genauigkeit und Präzision haben. Die Genauigkeit beschreibt die Streuung einer Messung, also wie gross die Varianz von einzelnen Messungen bei gleichbleibendem Messwert ist. Die Präzision gibt an, wie der systematische Fehler zum wahren Wert ist. Das übergeordnete Ziel dieser Filter ist es, eine möglichst genaue Schätzung des wahren Wertes anhand von fehlerbehafteten Sensordaten zu machen.

2.3.3.1.1 Grundprinzip

Der Alpha-Beta Filter basiert auf einem rekursiven Algorithmus, welcher im Wesentlichen aus zwei Phasen besteht. Die folgende Abbildung 7 zeigt den systematischen Aufbau:

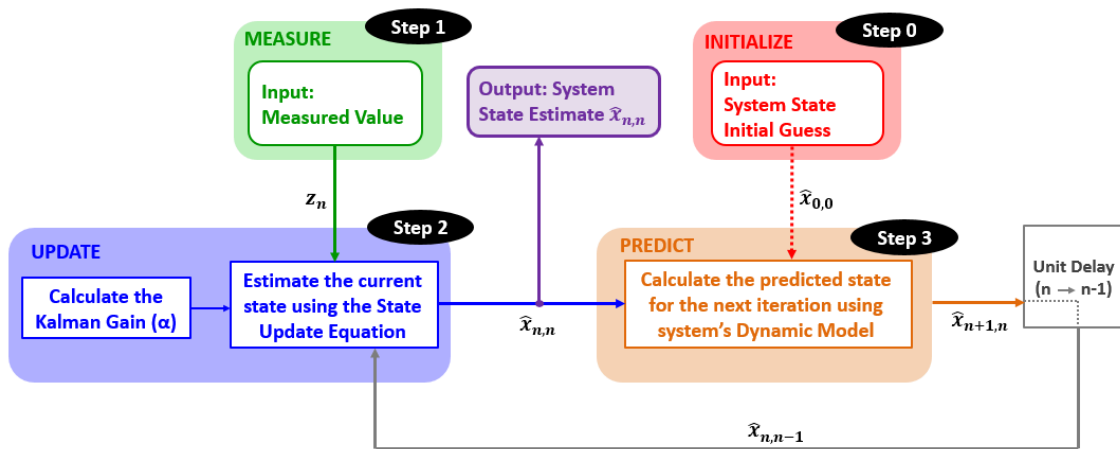


Abbildung 7: Schematischer Algorithmus Alpha-Beta Filter (Quelle: [14])

Die **predict**-Funktion (orange) schätzt anhand des Systemmodells und des momentanen States den darauffolgenden State des nächsten Zeitschritts $\hat{x}_{n+1,n}$. Die **update**-Funktion (blau) kombiniert den geschätzten State mit einer neuen Messung z_n und trifft eine Annahme für den tatsächlichen State $\hat{x}_{n,n}$. Die Differenz zwischen gemessenem Wert und Erwartungswert nennt sich *residual* oder *innovation*.

2.3.3.1.2 Beispiel: Statisches System

Ein einfaches Beispiel verdeutlicht die beschriebenen Prinzipien. Eine Waage misst wiederholt das Gewicht einer Person. Die Messwerte unterscheiden sich bei jeder Messung geringfügig aufgrund der begrenzten Messgenauigkeit. Diese Streuung führt zu Unsicherheiten in der Gewichtsbestimmung. Wiederholte Messungen reduzieren diese Unsicherheit. Bei normalverteilten Messwerten liefert der Mittelwert die beste Schätzung des wahren Gewichts.

Die nachfolgende Abbildung 8 zeigt den prinzipiellen Aufbau des Algorithmus.

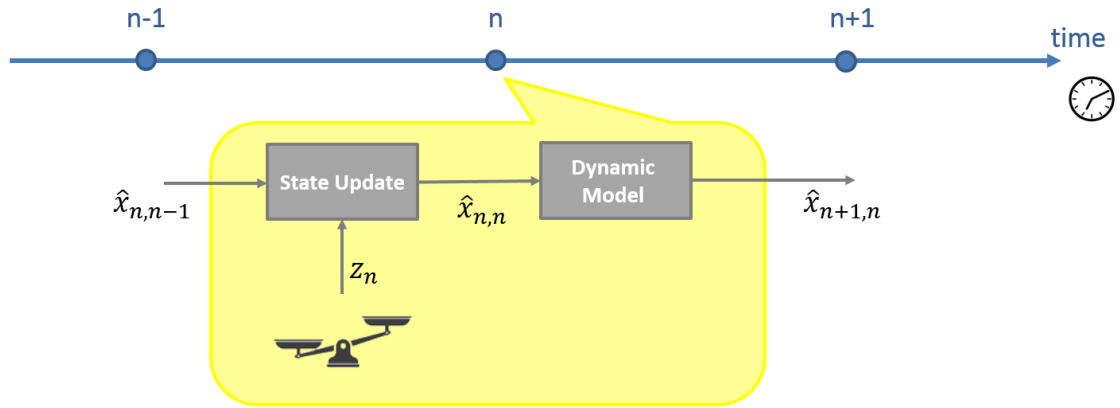


Abbildung 8: Diskrete Darstellung statisches System (Quelle: [14])

Es ist davon auszugehen, dass sich das Gewicht der Person zwischen den Messungen nicht ändert. Somit handelt es sich bei der Gewichtsmessung um ein statisches Systemmodell. Die Gleichung im Systemmodell nennt sich *State Extrapolation Equation* und ist wie folgt für dieses Beispiel definiert:

$$\hat{x}_{n+1,n} = \hat{x}_{n,n}$$

Der State Update-Schritt wird durch die *State Update Equation* definiert und schätzt den tatsächlichen Wert anhand der vorherigen Daten und des neu gemessenen Werts:

$$\hat{x}_{n,n} = \hat{x}_{n,n-1} + \alpha(z_n - \hat{x}_{n,n-1})$$

Wie aus der Formel ersichtlich, gewichtet der Wert α die Differenz der Messung und des geschätzten Werts. Im Falle einer Durchschnittsberechnung gilt also: $\alpha = \frac{1}{n}$.

- x verborgener State, welcher nicht direkt messbar ist
- z_n gemessener Wert zum Zeitpunkt n .
- $\hat{x}_{n,n}$ ist die Schätzung von x zum Zeitpunkt n (die Schätzung wird nach der Messung z_n erstellt).
- $\hat{x}_{n+1,n}$ ist die Schätzung des zukünftigen Zustands ($n+1$) von x . Die Schätzung wird zum Zeitpunkt n erstellt. $\hat{x}_{n+1,n}$ ist ein vorhergesagter oder extrapolierte Zustand.
- $\hat{x}_{n,n-1}$ ist die Schätzung von x zum Zeitpunkt $n-1$.

2.3.3.1.3 Parameter

Wie die Namensgebung des Filters verspricht, existiert ebenfalls ein Parameter β . Ein solcher findet Verwendung in der State Update Equation in zwei Dimensionen. Ebenfalls kann je nach Anwendungszweck auch ein dritter (γ) Parameter Verwendung finden. Möchte man den Filter für eine Standortbestimmung eines fahrenden Objekts verwenden, so würde sich das Systemmodell aus Position und Geschwindigkeit zusammensetzen. In diesem Falle lässt sich die State update Equation wie folgt definieren:

$$\hat{x}_{n,n} = \hat{x}_{n,n-1} + \alpha(z_n - \hat{x}_{n,n-1})$$

$$\dot{\hat{x}}_{n,n} = \dot{\hat{x}}_{n,n-1} + \beta \left(\frac{z_n - \hat{x}_{n,n-1}}{\Delta t} \right)$$

Die State Extrapolation Funktion bei gleichbleibender Geschwindigkeit lässt sich mit folgender Formel definieren:

$$x_{n+1} = x_n + \Delta t \dot{x}_n$$

$$\dot{x}_{n+1} = \dot{x}_n$$

Die Position ist abhängig von der Geschwindigkeit des Objekts. Somit ist relevant, dass die Geschwindigkeit in der Vorhersage der Position miteinbezogen wird.

2.3.3.2 Kalman Filter Prinzip

Der Kalman Filter für lineare Systeme ist nun eine abgeänderte Variante des Alpha-Beta Filters. Während die Prinzipien der Rekursion und der beiden Prozeduren *update* und *predict* bestehen bleiben, fügt der Kalman Filter die Gauss-Repräsentation von Messwerten ein. Die folgende Abbildung 9 zeigt exemplarisch eine Glockenkurve zur Repräsentation von Messwerten.

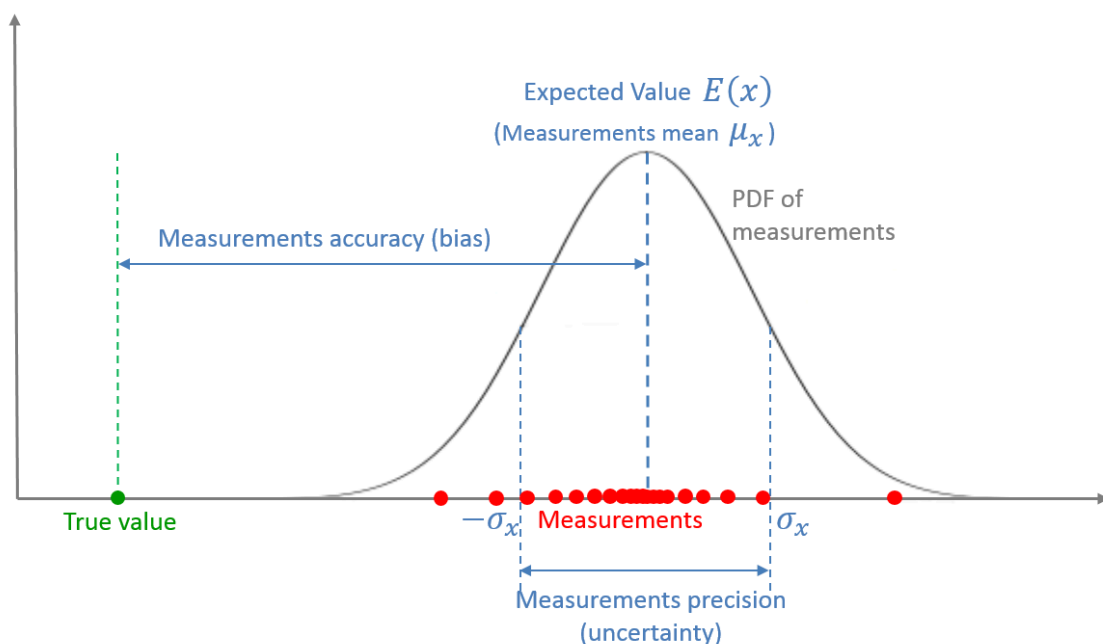


Abbildung 9: Gaussian-Glockenkurve (Quelle: [14])

Jeder Messwert und somit auch jeder State wird als *Erwartungswert* mit zugehöriger *Unsicherheit* betrachtet und als normalverteilt angenommen. Somit wird die Ungenauigkeit von Messwerten abgebildet und in den Berechnungen berücksichtigt.

Das schematische Grundprinzip des Kalman Filter lässt sich durch die integration der Gauss-Repäsentation der Messwerte in das Alpha-Beta Prinzip darstellen:

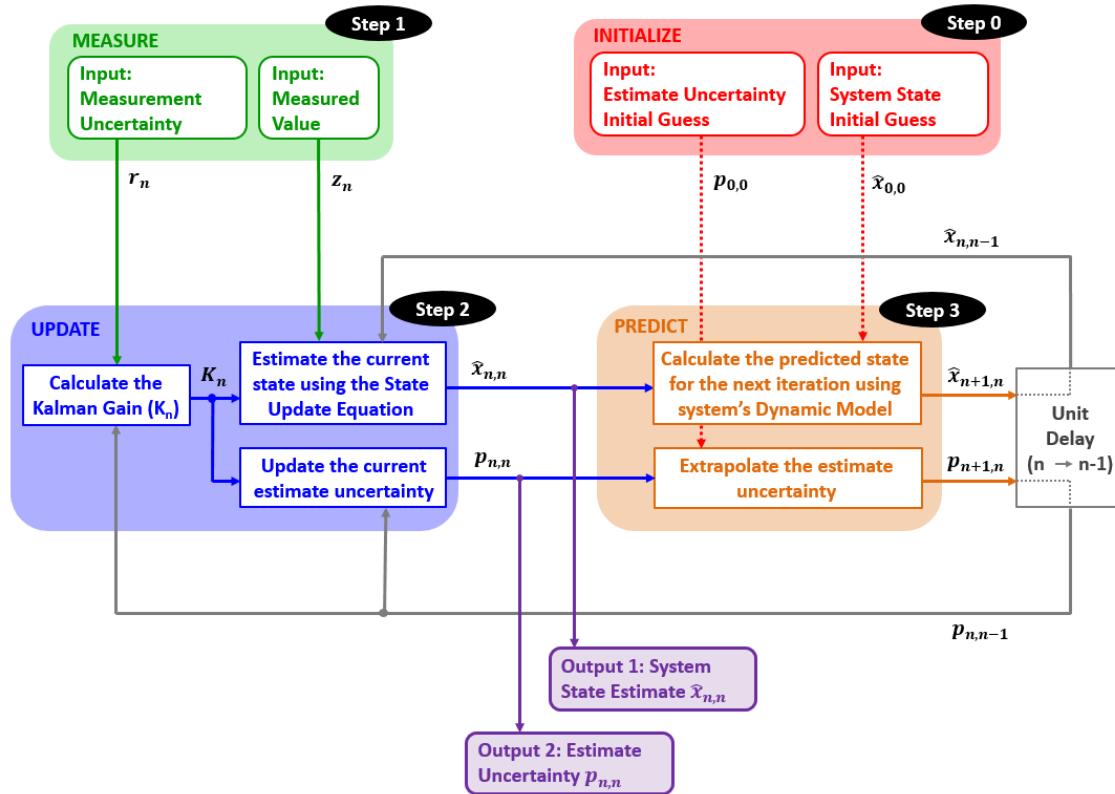


Abbildung 10: Schematischer Algorithmus des Kalman Filter (Quelle: [14])

Nebst der Doppelspurigkeit ändert sich schematisch gesehen gegenüber dem Alpha-Beta Filter nichts. Durch das Einführen der Varianz sollen jedoch einige bemerkenswerte Aspekte erläutert werden. Im folgenden sind die wesentlichen Schritte des in Abbildung 10 dargestellten Diagramms erklärt:

Step 0: Die Initialisierung des Kalman Filter (rot) erfordert das Bestimmen eines initialen States und einer zugehörigen Varianz. Die Wahl von p (Unsicherheit in x) definiert, wie schnell der Kalman Filter die Schätzung in Richtung der Messung verschiebt.

Step 1: Wie bei der initialen Schätzung von x muss auch bei der Messung (grün) eine Unsicherheit definiert werden. Im Falle eines Sensors kann es sein, dass die Varianz der Messung im Datenblatt ersichtlich ist. Ansonsten gilt hier, dass r_n geschätzt oder empirisch ermittelt werden muss.

Step 2: Der update-Schritt (blau) beinhaltet im Wesentlichen drei Berechnungen, die anschliessend genauer betrachtet werden. Mittels Kalman Gain und Residual wird eine Schätzung für den State $\hat{x}_{n,n}$ berechnet. Ebenfalls wird anhand des Kalman Gain die Unsicherheit für den geschätzten State berechnet.

Step 3: In der Predict-Phase (orange) wird aus dem letzten State und dem Systemmodell eine Vorhersage für den nächsten State getroffen.

Da die wesentlichen Berechnungen in den beiden Schritten *update* und *predict* stattfinden, werden diese nochmals genauer beleuchtet. Dabei wird nun die multivariate Notation für Berech-

nungen eingeführt. Diese verwendet Vektoren und Matrizen. Die meisten Systeme, in denen ein Kalman Filter zum Einsatz kommt, sind mehrdimensional. Eine univariate Schreibweise wäre sehr aufwändig und unübersichtlich.

2.3.3.3 Predict

Der Predict-Schritt extrapoliert sowohl den geschätzten Zustand als auch die zugehörige Unsicherheit vom aktuellen Zeitpunkt zum nächsten, basierend auf dem mathematischen Modell (Systemmodell $\mathbf{F}$). Das mathematische Modell beschreibt das zu messende System und dessen physikalisches Verhalten.

Folgende Operationen werden in der Predict-Phase durchgeführt:

Die Extrapolation des Zustands ist wie folgt definiert:

$$\hat{\mathbf{x}}_{n+1,n} = \mathbf{F}\hat{\mathbf{x}}_{n,n} + \mathbf{G}\mathbf{u}_n$$

$\hat{\mathbf{x}}_{n+1,n}$	vorhergesagter Zustandsvektor zum Zeitpunkt $n + 1$, basierend auf Informationen bis n
$\mathbf{F}$	System Matrix (State Transition Matrix), beschreibt die Systemdynamik
$\hat{\mathbf{x}}_{n,n}$	geschätzter Zustandsvektor zum Zeitpunkt n nach dem Update
$\mathbf{G}$	Control Matrix, verknüpft Inputvektor mit dem Zustand
$\mathbf{u}_n$	Inputvektor zum Zeitpunkt n

Für die Extrapolation der Unsicherheit in den Zustand gilt:

$$\mathbf{P}_{n+1,n} = \mathbf{F}\mathbf{P}_{n,n}\mathbf{F}^T + \mathbf{Q}$$

$\mathbf{P}_{n+1,n}$	vorhergesagte Kovarianzmatrix zum Zeitpunkt $n + 1$, repräsentiert die Unsicherheit der Vorhersage
$\mathbf{P}_{n,n}$	Kovarianzmatrix zum Zeitpunkt n nach dem Update
$\mathbf{F}$	System Matrix (State Transition Matrix), beschreibt die Systemdynamik
$\mathbf{Q}$	Prozesskovarianzmatrix, modelliert die Unsicherheit im Systemmodell

2.3.3.4 Update

Der Update-Schritt ist die zweite Phase des Kalman-Filters und korrigiert die Vorhersage aus dem Predict-Schritt mit den aktuellen Sensormessungen. Dieser Schritt erfolgt in drei Berechnungen:

Kalman Gain Berechnung: Zunächst wird der optimale Kalman Gain berechnet, der bestimmt, wie stark die neue Messung gegenüber der Vorhersage gewichtet wird. Ein hoher Gain bedeutet, dass der Messung mehr vertraut wird, ein niedriger Gain bevorzugt die Vorhersage durch das Systemmodell. Der Kalman Gain berechnet sich wie folgt:

$$\mathbf{K}_n = \mathbf{P}_{n,n-1}\mathbf{H}^T(\mathbf{H}\mathbf{P}_{n,n-1}\mathbf{H}^T + \mathbf{R}_n)^{-1}$$

$\mathbf{K}_n$	Kalman Gain Matrix, bestimmt die Gewichtung zwischen Vorhersage und Messung
----------------	---

$P_{n,n-1}$	Vorhergesagte Kovarianzmatrix (Unsicherheit der Vorhersage)
H	Observation Matrix, bildet Messung auf den Zustand ab
R_n	Messrauschkovarianzmatrix, beschreibt die Unsicherheit der Sensoren

Zustandskorrektur: Der vorhergesagte Zustand wird mit dem gewichteten Residuum korrigiert. Dies führt zu einer optimalen Schätzung des aktuellen Systemzustands.

$$\hat{\mathbf{x}}_{n,n} = \hat{\mathbf{x}}_{n,n-1} + \mathbf{K}_n(\mathbf{z}_n - \mathbf{H}\hat{\mathbf{x}}_{n,n-1})$$

$\hat{\mathbf{x}}_{n,n}$	korrigierter Zustandsvektor nach Einbeziehung der Messung
$\hat{\mathbf{x}}_{n,n-1}$	vorhergesagter Zustandsvektor vor der Korrektur
$\mathbf{z}_n$	Messvektor zum Zeitpunkt n
$(\mathbf{z}_n - \mathbf{H}\hat{\mathbf{x}}_{n,n-1})$	Residuum (Innovation), Differenz zwischen Messung und Vorhersage

Unsicherheitsreduktion: Die Kovarianzmatrix wird aktualisiert durch die Kombination von Vorhersage und Messung. Dadurch verringert sich die Unsicherheit im State.

$$P_{n,n} = (\mathbf{I} - \mathbf{K}_n\mathbf{H})P_{n,n-1}(\mathbf{I} - \mathbf{K}_n\mathbf{H})^T + \mathbf{K}_n\mathbf{R}_n\mathbf{K}_n^T$$

$P_{n,n}$	korrigierte Kovarianzmatrix nach dem Update
$\mathbf{I}$	Einheitsmatrix

Das Ergebnis ist eine verbesserte Zustandsschätzung mit reduzierter Unsicherheit, die als Ausgangspunkt für den nächsten Predict-Schritt dient.

2.3.3.5 Tuning-Parameter

Anhand eines einfachen Beispiels werden in diesem Abschnitt die Einstellparameter des Filters genauer betrachtet. Dazu dient eine simulierte Spannungsmessung mit einstellbarer Varianz.

2.3.3.5.1 Die Prozesskovarianzmatrix $\mathbf{Q}$

Die Prozesskovarianzmatrix $\mathbf{Q}$ wird in der Predict-Phase verwendet, um die Kovarianz des Systemmodells zu beeinflussen. Das folgende Experiment soll verdeutlichen, welchen Einfluss die Matrix auf den Filter hat.

In beiden nachfolgenden Abbildung 11 und Abbildung 12 ist ersichtlich, wie der Kalman Filter rasch zum wahren Wert (graue Linie) konvergiert. Die Varianz wird mit einem hellblauen Bereich um den State dargestellt. Durch den höheren Wert von $\mathbf{Q}$ in Abbildung 12 ist die Unsicherheit in das Modell erhöht, wodurch die Messdaten vertrauenswürdiger sind. In Abbildung 11 hingegen ist das Vertrauen in das Systemmodell grösser, wodurch eine bessere Glättung der Messdaten stattfindet.

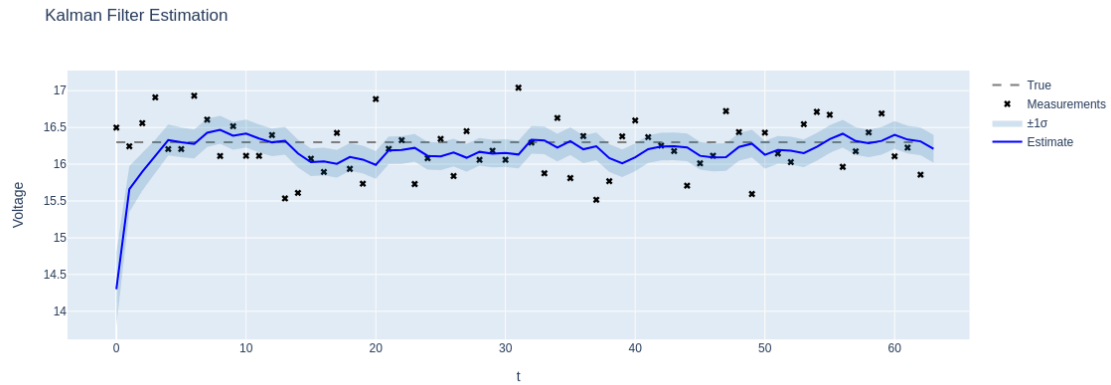


Abbildung 11: Kalman Filter mit $Q = 0.01$, $R = 0.4$

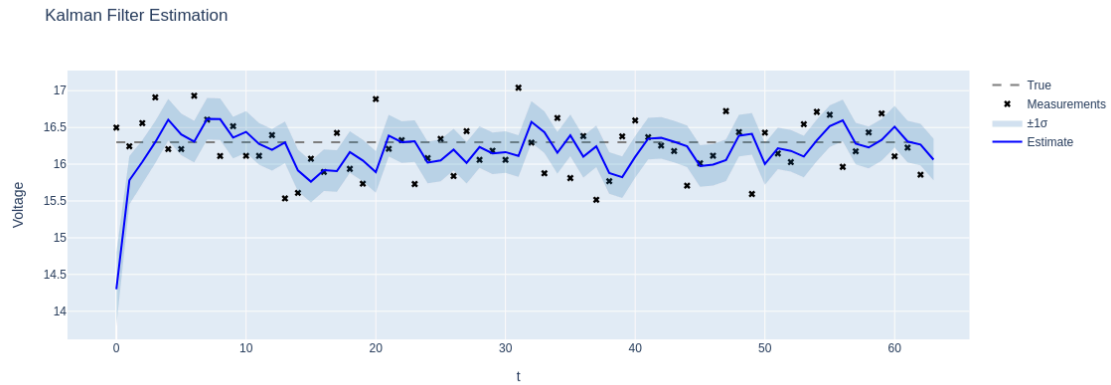


Abbildung 12: Kalman Filter mit $Q = 0.04$, $R = 0.4$

Um auf effektive Änderungen im System angemessen reagieren zu können, darf die Matrix Q nicht zu klein gewählt werden. Hier gilt es einen Trade-off zwischen Glättung und Reaktion auf Veränderungen im System zu finden.

2.3.3.5.2 Die Messkovarianzmatrix R

Die Messkovarianzmatrix beschreibt die Varianz von Messungen. Im System beeinflusst dieser Wert das Vertrauen in die Messwerte und wird in der Update-Phase verwendet. Die folgenden Abbildung 13 und Abbildung 14 zeigen die Schätzungen für das Spannungssignal mit unterschiedlichen Messkovarianzmatrizen. Abbildung 13 stellt den Schätzwert mit geringem Messvarianz-Wert dar. Dadurch entsteht ein grösseres Vertrauen in die Messdaten, sodass die Schätzung des Kalman-Filters jeweils nahe den Messsignalen resultiert.

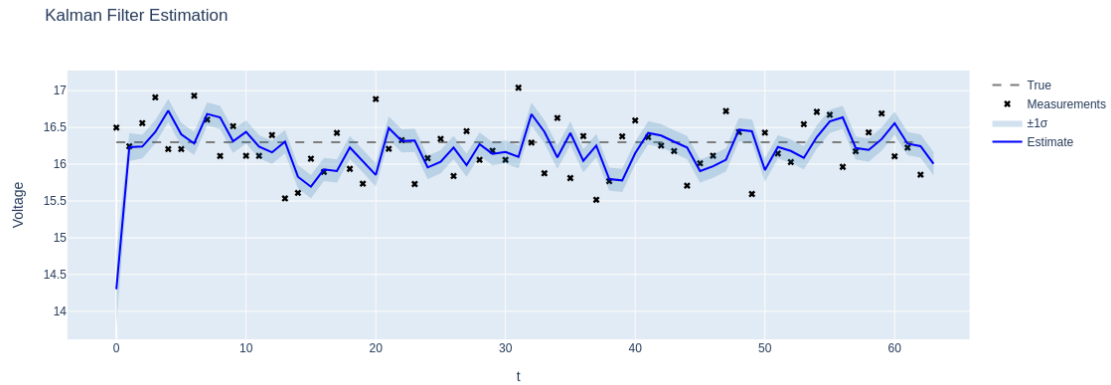


Abbildung 13: Kalman Filter mit $Q = 0.04$, $R = 0.2$

Der zweite Versuch in Abbildung 14 wurde mit einer deutlich höheren Messvarianz durchgeführt. Dabei fällt auf, dass sich der geschätzte Wert nur langsam dem wahren Wert annähert. Dies ist auf die hohe Unsicherheit der Messdaten zurückzuführen, wodurch ein geglättetes Signal entsteht.

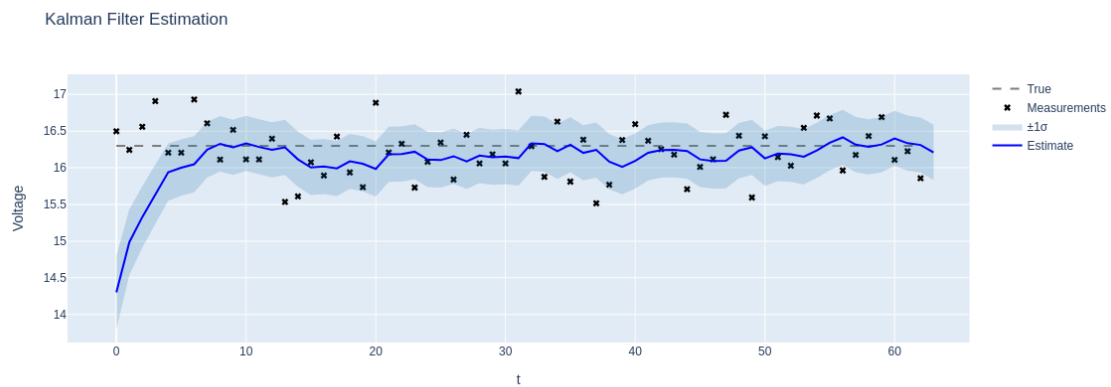


Abbildung 14: Kalman Filter mit $Q = 0.04$, $R = 0.8$

2.3.3.5.3 Kalman Gain

Der Kalman Gain ist ein zentrales Element im Kalman Filter und ist verantwortlich für die optimale Schätzung des Zustandes. Dabei kann er ein Wert zwischen 0 und 1 annehmen, welcher anhand der beiden Unsicherheiten aus Messung und Schätzung gebildet wird. Je näher der Kalman Gain an eins liegt, desto mehr wird der Messung vertraut, während ein niedrigerer Wert die Schätzung stärker gewichtet. Die Abbildung 15 visualisiert den Vorgang.

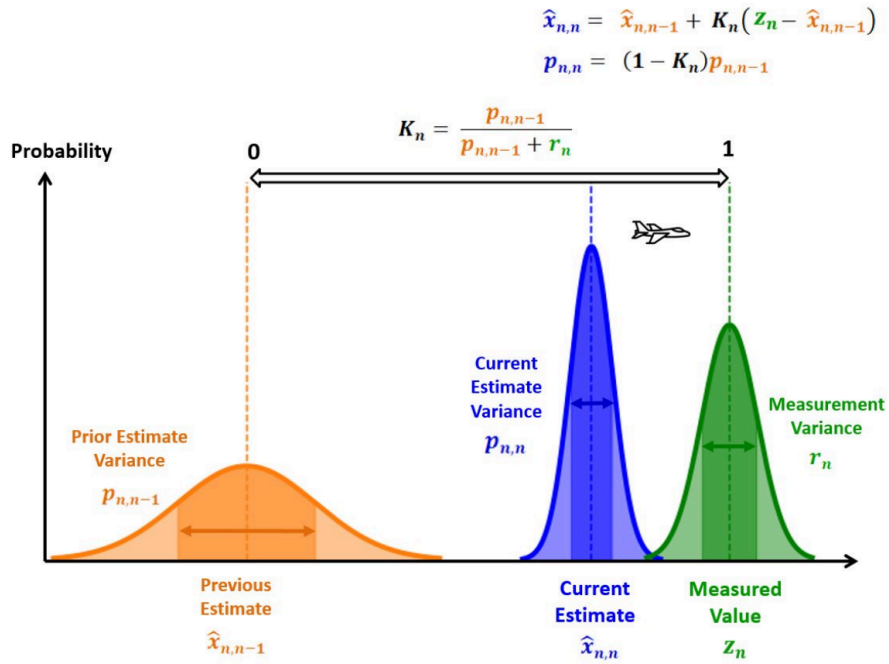


Abbildung 15: Berechnung Kalman Gain (Quelle: [14])

Der Kalman Gain berechnet sich anhand der Varianz der Messung (grün) und der Varianz des geschätzten Wertes (orange). Das Resultat der Berechnung ist der geschätzte Wert des Filters (blau).

2.3.3.6 Zusammenfassung

Die folgende Abbildung 16 zeigt eine Zusammenfassung der Gleichungen:

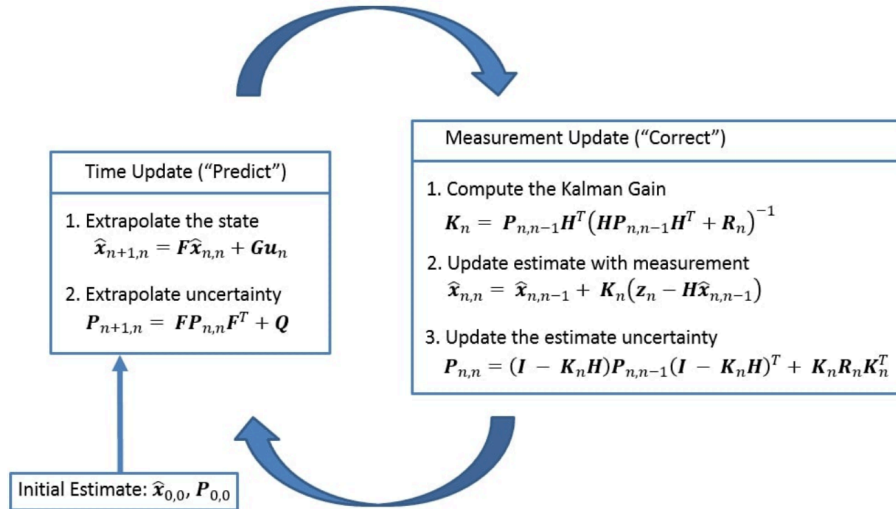


Abbildung 16: Gleichungen Kalman Filter Algorithmus (Quelle: [14])

Dieser Algorithmus ist auf lineare Systeme beschränkt. Das reicht insbesondere für das Abschätzen des SOC in Batterien nicht aus. Wie bereits erwähnt, ist die Batterie ein nichtlineares System. Aus diesem Grund wird im Folgenden eine Erweiterung des bisherigen Konzepts vorgestellt.

2.3.4 Extended Kalman Filter

Der Kalman Filter wurde ursprünglich für lineare Systeme entwickelt. Weil die meisten realen Systeme jedoch von nicht-linearer Natur sind, haben Forscher am NASA Research Center den Extended Kalman Filter (EKF) entwickelt [15]. Die Hauptidee hinter dem EKF ist das Linearisieren der nicht-linearen Gleichungen am Arbeitspunkt.

2.3.4.1 Motivation

Der ursprüngliche Kalman-Filter ist an zwei Stellen von linearen Systemen abhängig. Zum einen betrifft dies das *State to Measurement Modell*, welches durch die Matrix $\mathbf{H}$ definiert ist und im Update-Schritt Verwendung findet. Andererseits betrifft es die Matrix $\mathbf{F}$, die das Systemmodell repräsentiert und im Predict-Schritt angewendet wird. Problematisch ist dabei die Berechnung der Normalverteilung. Der Kalman-Filter setzt voraus, dass alle Verteilungen normal (Gauss) sind. Diese Bedingung ist jedoch nicht mehr erfüllt, wenn eine Normalverteilung mit einem nichtlinearen System verrechnet wird. Da das Lösungsprinzip bei beiden Gleichungen dasselbe ist, wird im Folgenden nur die Behandlung der Messmatrix $\mathbf{H}$ detailliert beschrieben.

2.3.4.2 Nichtlineare Messmatrix $\mathbf{H}$

Um die Problematik der nicht-linearen Systeme aufzuzeigen, dient ein Beispiel aus dem Buch *Kalman Filter from the Ground* [14]. Ziel ist es, die Höhe eines Luftballons zu schätzen, indem aus einer Entfernung d den Winkel θ zur Ballongondel gemessen wird. Die Abbildung 17 verdeutlicht das System.

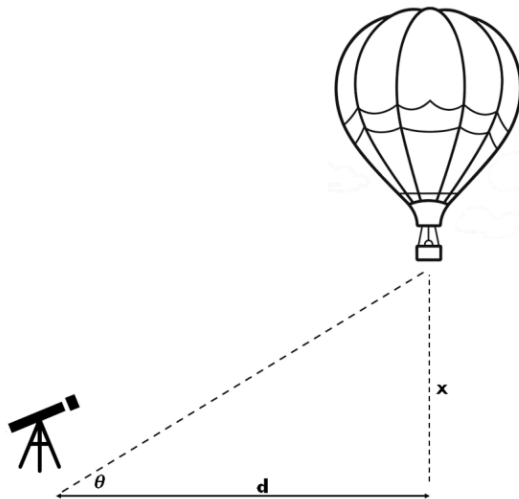


Abbildung 17: Ballonmessung (Quelle: [14])

Die Berechnung der Höhe ist durch $x = d * \tan(\theta)$ definiert. Die Messgleichung des Kalman Filters für ein lineares System lautet $z_n = \mathbf{H}x_n + v_n$.

Die Abbildung 18 verdeutlicht die Verzerrung der Normalverteilung bei einem nicht-linearen System.

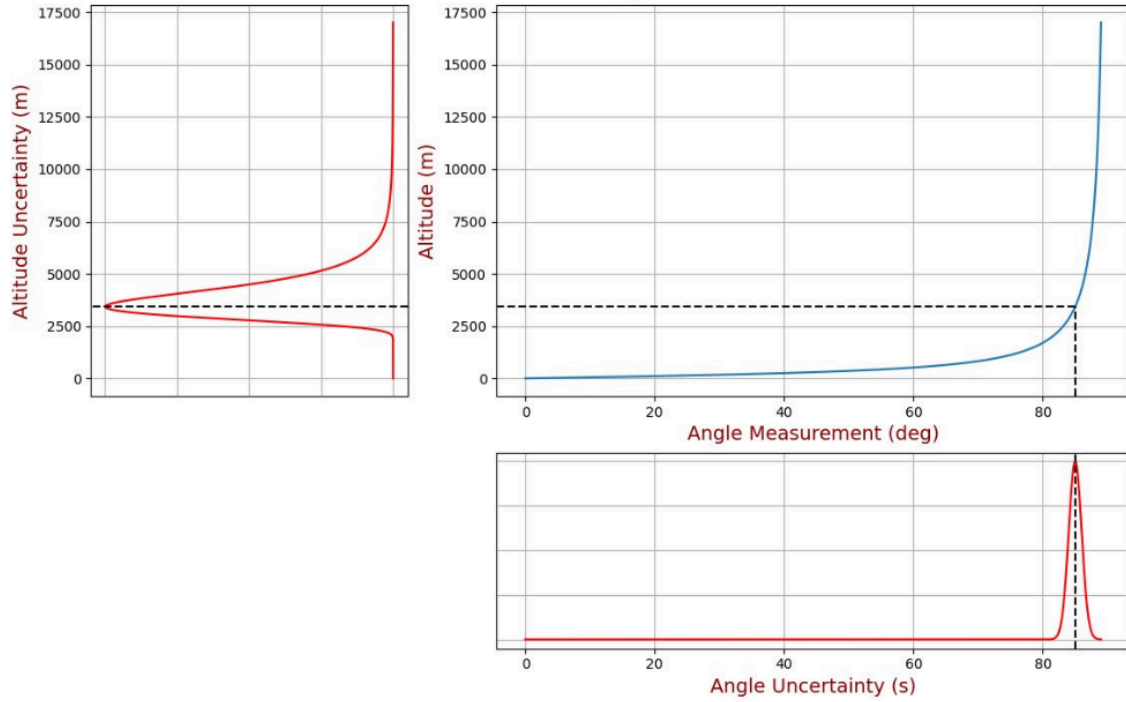


Abbildung 18: Nichtlineares System (Quelle: [14])

Die Varianzberechnung der Höhe ist durch die Matrix $\mathbf{H}$ inkorrekt. Für die Lösung dieses Problems gibt es verschiedene Ansätze. Die bekanntesten dabei sind der Extended Kalman Filter und der Uncented Kalman Filter. Da in dieser Arbeit der EKF verwendet wird, betrachten wir dessen Lösungsansatz.

Wie bereits erwähnt wird das Problem durch Linearisieren gelöst. Dazu ändert sich die Berechnung von z_n wie folgt:

$$z_n = h(x_n)$$

Für dieses Beispiel gilt:

$$z_n = \theta = \tan^{-1} \frac{x_n}{d}$$

$$h(x_n) = \theta = \tan^{-1} \frac{x_n}{d}$$

Zur Linearisierung des Systems wird differenziert: Die Ableitung des inversen Tangens lautet wie folgt:

$$\frac{d \tan^{-1}(x)}{d_x} = \frac{1}{1 + x^2}$$

Durch Einsetzen von $\frac{x_n}{d}$ ergibt sich folgende Gleichung:

$$\frac{d \tan^{-1}\left(\frac{x_n}{d}\right)}{d_x} = \frac{1}{1 + \left(\frac{x_n}{d}\right)^2}$$

Nun wird nach x abgeleitet durch Anwenden der Kettenregel: $h'(x) = u'(v(x)) * v'(u)$
wobei: $u = \tan^{-1}(x)$ und $v = \frac{x}{d}$ sind.

$$\frac{d \tan^{-1}\left(\frac{x}{d}\right)}{d_x} = \frac{1}{1 + \left(\frac{x}{d}\right)^2} \cdot \frac{1}{d}$$

Dadurch ergibt sich die Beobachtungsmatrix $\mathbf{H}$ für dieses System.

$$\frac{dh(x)}{d} = \frac{d}{d^2 + x^2}$$

Durch Anwenden der Linearisierung am Arbeitspunkt erreicht man nun eine korrekte Normalverteilung der Höhen-Schätzung, wie in Abbildung 19 dargestellt.

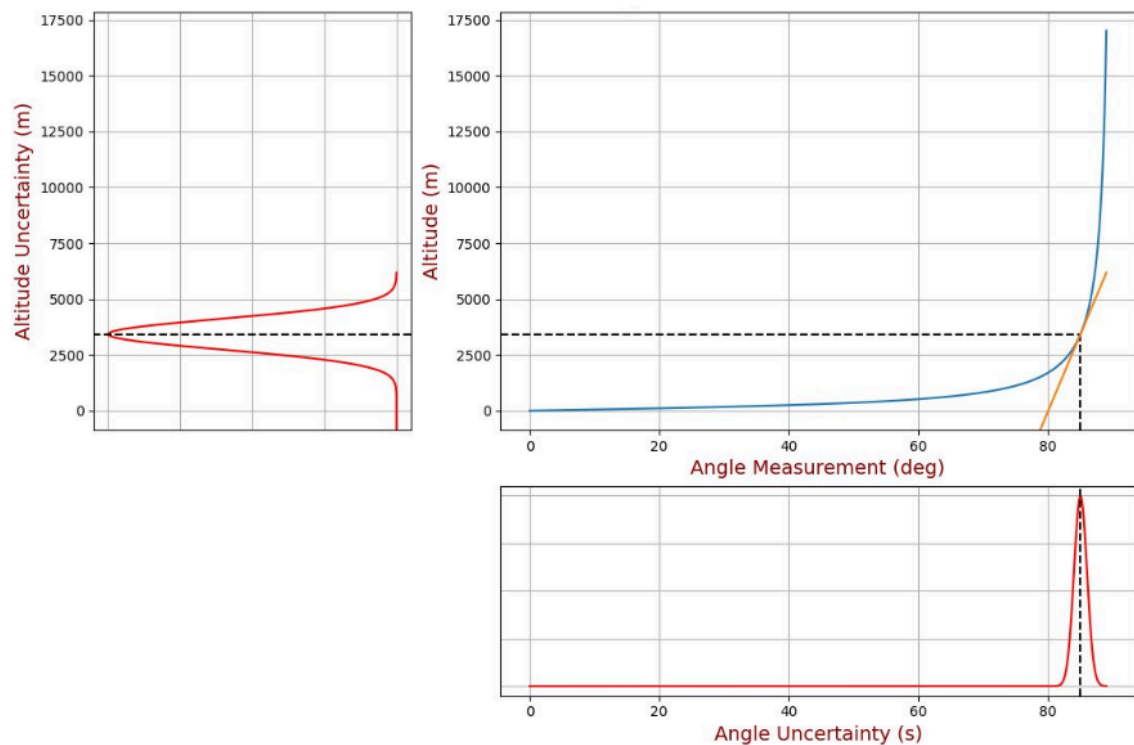


Abbildung 19: Linearisiertes System (Quelle: [14])

2.3.4.3 Projektion multivariater Unsicherheit

Für viele Systeme reicht es nicht, nur eine Dimension zu berücksichtigen. Aus diesem Grund verwendet man die multivariate Form. Möchte man zum Beispiel den Ort sowie die Geschwindigkeit eines Objekts schätzen, beeinflusst die Unsicherheit in der Geschwindigkeitsschätzung auch die Unsicherheit, wo sich dieses Objekt befindet. Diese Zusammenhänge in mehreren Dimensionen nennt man Kovarianzen. Sie werden in nichtlinearen Systemen ebenfalls durch die partiellen Ableitungen verrechnet. Um dies in allen Dimensionen zu ermöglichen, wird jeweils eine Jacobi-Matrix erstellt. Die Systemmatrix $\mathbf{F}$ und die Messmatrix $\mathbf{H}$ werden beim EKF als solche abgebildet.

2.3.5 Zusammenfassung

Für eine bessere Übersicht stellt die nachfolgende Tabelle 1 die Gleichungen des linearen Kalman Filters denjenigen des Extended Kalman Filters gegenüber.

	Name	LKF Formeln	EKF Formeln
Predict	State Extrapolation	$\hat{x}_{n+1,n} = F\hat{x}_{n,n} + Gu_n$	$\hat{x}_{n+1,n} = f(\hat{x}_{n,n}) + Gu_n$
	Covariance Extrapolation	$P_{n+1,n} = FP_{n,n}F^T + Q$	$P_{n+1,n} = \frac{\partial f}{\partial x}P_{n,n}\left(\frac{\partial f}{\partial x}\right)^T + Q$
Update	State Update	$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n(z_n - H\hat{x}_{n,n-1})$	$\hat{x}_{n,n} = \hat{x}_{n,n-1} + K_n(z_n - h(\hat{x}_{n,n-1}))$
	Covariance Update	$P_{n,n} = (I - K_nH)P_{n,n-1} \times (I - K_nH)^T + K_nR_nK_n^T$	$P_{n,n} = (I - K_n\frac{\partial h}{\partial x})P_{n,n-1} \times (I - K_n\frac{\partial h}{\partial x})^T + K_nR_nK_n^T$
	Kalman Gain	$K_n = P_{n,n-1}H^T \times (HP_{n,n-1}H^T + R_n)^{-1}$	$K_n = P_{n,n-1}\left(\frac{\partial h}{\partial x}\right)^T \times \left(\frac{\partial h}{\partial x}P_{n,n-1}\left(\frac{\partial h}{\partial x}\right)^T + R_n\right)^{-1}$

Tabelle 1: Vergleich der Kalman-Filter Gleichungen: Linear (LKF) vs. Extended (EKF)

2.4 Robot Operating System (ROS)

Damit das Messsystem effektiv genutzt werden kann, muss es in der Lage sein, aktuelle Daten des Roboters abzurufen. Das Messsystem soll in die bestehende Plattform integriert werden, die über ROS gesteuert wird.

Das Robot Operating System kurz ROS ist ein modulares Software-Framework für die Entwicklung von Robotikanwendungen. Trotz des Namens handelt es sich nicht um ein klassisches Betriebssystem, sondern um eine Sammlung von Libraries, Werkzeugen und Konventionen. ROS wurde ursprünglich 2007 am Stanford Artificial Intelligence Laboratory entwickelt und wird heute hauptsächlich von der Open Source Robotics Foundation gepflegt [16].

2.4.1 Architektur

Die Architektur von ROS basiert auf einem verteilten System aus kommunizierenden Prozessen, die als Nodes (Knoten) bezeichnet werden. Jeder Node ist ein eigenständiges Programm, das eine spezifische Aufgabe erfüllt, wie beispielsweise das Auslesen von Sensordaten, die Pfadplanung oder die Motorsteuerung.

Die Kommunikation zwischen den Nodes erfolgt über ein Publish-Subscribe-System mittels sogenannter Topics. Ein Node kann Daten auf einem Topic veröffentlichen (publish), während andere Nodes diese Daten abonnieren können (subscribe).

2.4.2 Nachrichten und Datentypen

ROS definiert standardisierte Nachrichtenformate für den Datenaustausch zwischen Nodes. Beispiele sind Sensordaten wie Laser-Scans, Batteriedaten oder auch primitive Datentypen. Die Standardisierung der Nachrichtenformate ermöglicht eine einfache Integration verschiedener Hardware-Komponenten und Software-Module, unabhängig vom Hersteller oder der spezifischen Implementierung.

3 Umsetzung

Der Hauptteil von diesem Kapitel handelt von der Umsetzung der Extended Kalman Filter für die Schätzung des *State of Charge* und *State of Health*. Dazu werden die einzelnen Schritte zur Erstellung der jeweiligen Systemmodelle beschrieben. Anschliessend wird aufgezeigt, welche mathematischen Gleichungen im Kalman Filter zum Einsatz kommen. Nach der Implementierung folgt ein Funktionstest, um die Korrektheit zu verifizieren.

Im nächsten Schritt wird aufgezeigt, wie die entwickelten Filter in das bestehende System integriert werden. Zu diesem Zweck wird ein zusätzlicher Sensor verbaut, welcher die erforderlichen Messwerte für die Berechnungen bereitstellt.

Der Schlussabschnitt untersucht den Energieverbrauch des Systems. Dabei werden verschiedene Analysen durchgeführt, um anschliessend ein Energieprofil zu erstellen. Darauf aufbauend lassen sich unter anderem Reichweitenabschätzungen durchführen. Der letzte Teil befasst sich mit dem Laden des Systems und den damit verbundenen Herausforderungen.

3.1 SOC - Extended Kalman Filter

Im Folgenden ist die Entwicklung eines Extended Kalman Filters zur SOC-Schätzung einer Blei-Säure Batterie beschrieben. Der Vorgang basiert auf dem Matlab-Beispiel *Explore Techniques to Estimate Battery State of Charge* [17]. Die Entwicklung des Kalman Filters beinhaltet im Wesentlichen die Erstellung der Systemmatrix $\mathbf{F}$ sowie der Messmatrix $\mathbf{H}$.

3.1.1 Systemmodell

Essentiell für die Dimensionierung eines SOC Kalman Filters ist ein Batterie spezifisches Systemmodell. Da jede Batterie einzigartig ist und eine eigene Charakteristik hat, gibt es kein generell anwendbares Modell. Das Systemmodell ist in diesem Falle ein Ersatzschaltkreis der Batterie, auch *Equivalent Circuit* oder *Thevenin Model* genannt.

3.1.1.1 Equivalent Circuit Model

Das Equivalent Circuit Model (ECM) beschreibt das dynamische Verhalten des gesamten Batterie-Blocks. Ein ECM besteht aus einer Spannungsquelle E_m , einem Serienwiderstand R_0 sowie einem oder mehreren parallel geschalteten RC-Gliedern, die in Serie geschaltet sind.

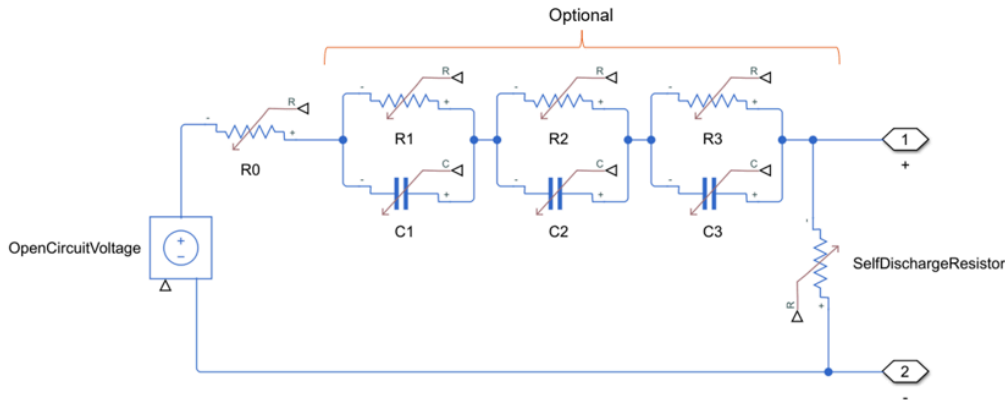


Abbildung 20: Equivalent Circuit Model (Quelle: [17])

Diese RC-Glieder modellieren verschiedene elektrochemische Zeitkonstanten und Diffusionsprozesse innerhalb der Batteriezelle [18]. Das Modell kann zusätzlich einen Zweig zur Selbstentladung und Ladeverluste enthalten, welcher durch einen *SelfDischargeResistor* definiert ist. Auf diesen wird in dieser Arbeit jedoch verzichtet.

Die Batteriespannung stabilisiert sich bei wechselnder Strombelastung erst nach gewissen Zeit. Um dieses Verhalten im System-Model nachzubilden, werden RC-Gliedern eingesetzt. Die Kondensatoren weisen durch ähnliche physikalische Vorgänge vergleichbare Effekte auf. Durch den parallelen Widerstand sowie die Kapazität des Kondensators lässt sich definieren, wie schnell sich der Kondensator lädt, resp. entlädt. Die Zeitkonstante τ berechnet sich wie folgt:

$$\tau = R_C \cdot C$$

Nach ungefähr $5 \cdot \tau$ ist der Kondensator vollständig geladen. Der Spannungsverlauf am Kondensator lässt sich durch die folgenden Formeln beschreiben [19].

Für die Entladung gilt:

$$u_{C(t)} = U_0 \cdot e^{-\frac{t}{\tau}}$$

Für die Ladung gilt:

$$u_{C(t)} = U_0 \cdot \left(1 - e^{-\frac{t}{\tau}}\right)$$

Die folgende Abbildung 21 zeigt vereinfacht den Spannungsverlauf an einer Batterie nach einem Stromsprung.

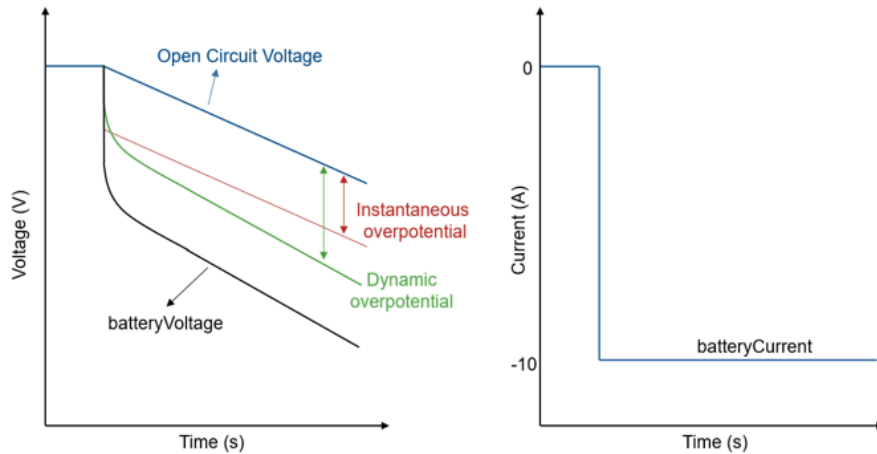


Abbildung 21: Spannungsverlauf bei Stromsprung (Quelle:[17])

Das *instantaneous overpotential* wird durch den Innenwiderstand R_0 definiert. Dieser verursacht einen linearen Spannungsabfall durch den fließenden Strom. Das *dynamic overpotential* wird durch die RC-Glieder bestimmt. Wie man aus der Grafik erkennen kann, wird die effektive Entladespannung erst nach einer gewissen Zeit erreicht.

3.1.1.2 Pulse Discharge Test

Der *hybrid pulse power characterization*, kurz HPPC-Test, ist eine Möglichkeit das ECM einer Batterie zu entwickeln. Dazu wird die Batterie wie folgt schrittweise entladen. Nach einer Belastung mit konstantem Strom folgt eine Ruhephase. Die Dauer der Ruhephase muss so gewählt werden, dass die Spannung auf das Level der OCV ansteigen kann. Dies wird wiederholt, bis die Batterie ihre Entladeschluss-Spannung erreicht hat. Während des Tests wird die Spannung an den Batterieanschlüssen gemessen. Die Dauer und Stromstärke der Belastung muss so gewählt werden, dass maximal 5% der Gesamtkapazität pro Puls entladen werden. Somit sind mindestens 20 Entladevorgänge notwendig [20], [21].

3.1.1.3 Testaufbau

Für die Durchführung des Tests wird eine Stromsenke verwendet. Eine Stromsenke ist ein elektronischer Widerstand, der dafür sorgt, dass bei variierender Spannung ein konstanter Strom fließt. Dies ist notwendig, da sich mit zunehmender Entladung der Innenwiderstand sowie die Charakteristik der Batterie verändert. Getestet wurden die beiden im Roboter eingesetzten Batterien in Parallelschaltung, so dass sich eine gesamte Nennkapazität von 44 Ah ergibt. Auf Abbildung 22 sieht man die Stromsenke (blau) und die beiden Akkumulatoren in Parallelschaltung.

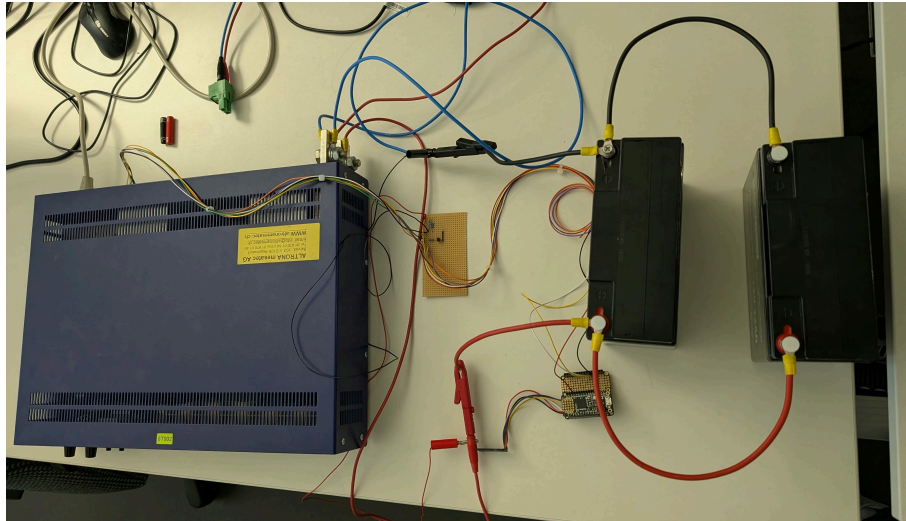


Abbildung 22: Test Aufbau

Mittels Mikrocontroller wird die Stromstärke über einen Analog/Digital-Ausgang eingestellt. Ein- und Ausschalten erfolgt über einen Transistor, der von einem Digitalausgang gesteuert wird. Der folgende Schaltkreis auf Abbildung 23 zeigt die Ansteuerung der Stromsenke.

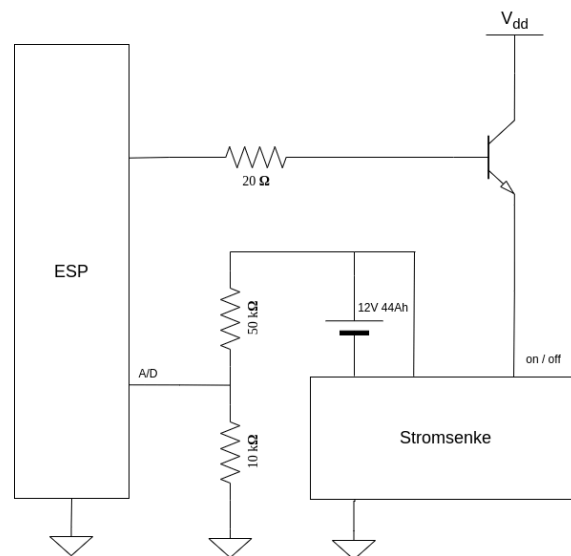


Abbildung 23: Vereinfachter Schaltkreis des Testaufbau

Die Belastung wird über einen Transistor aktiv geschaltet, während die Spannung über einen Spannungsteiler an der Batterie gemessen wird [22].

3.1.1.4 Testdaten

Die Abbildung 24 stellt den Spannungsverlauf während des Tests dar. Nebst der Spannung sind Zeit sowie der Lade- und Entladestatus aufgenommen worden.

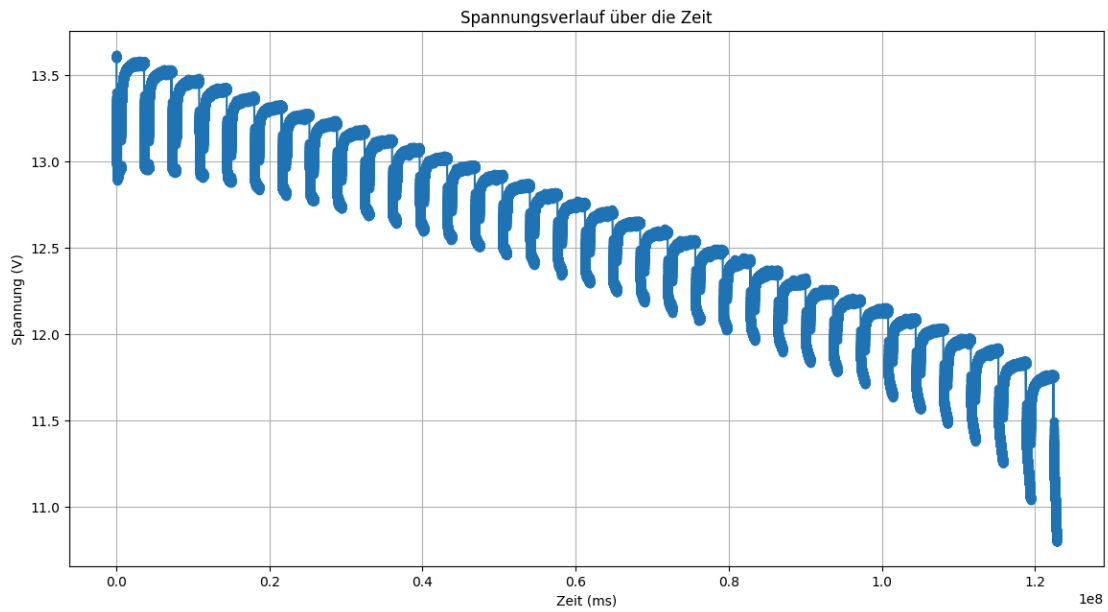


Abbildung 24: HPPC Rohdaten

Über einen Zeitraum von 34.1 Stunden sind über 1.1 M Datenpunkte aufgenommen worden, um den Spannungsverlauf möglichst präzise aufzuzeichnen. Der Spannungseinbruch wird jeweils durch die Belastung von 7.2 A verursacht. Nach 10 Minuten Entladen folgt eine Ruhephase von 50 Minuten, in der sich die Spannung bis zur OCV-Spannung regeneriert [18]. Die Aufnahme dieser Daten bilden die Grundlage für die Parameterdetektion des ECM mittels Matlab.

3.1.1.5 Batterie-Charakterisierung

Matlab bietet für die Berechnung der ECM-Elemente die entsprechenden Werkzeuge. Mit dem `Battery.PulseSequence`-Objekt lassen sich in einem ersten Schritt die einzelnen Entlade-Pulse identifizieren. Abbildung 25 veranschaulicht die detektierten Pulse.

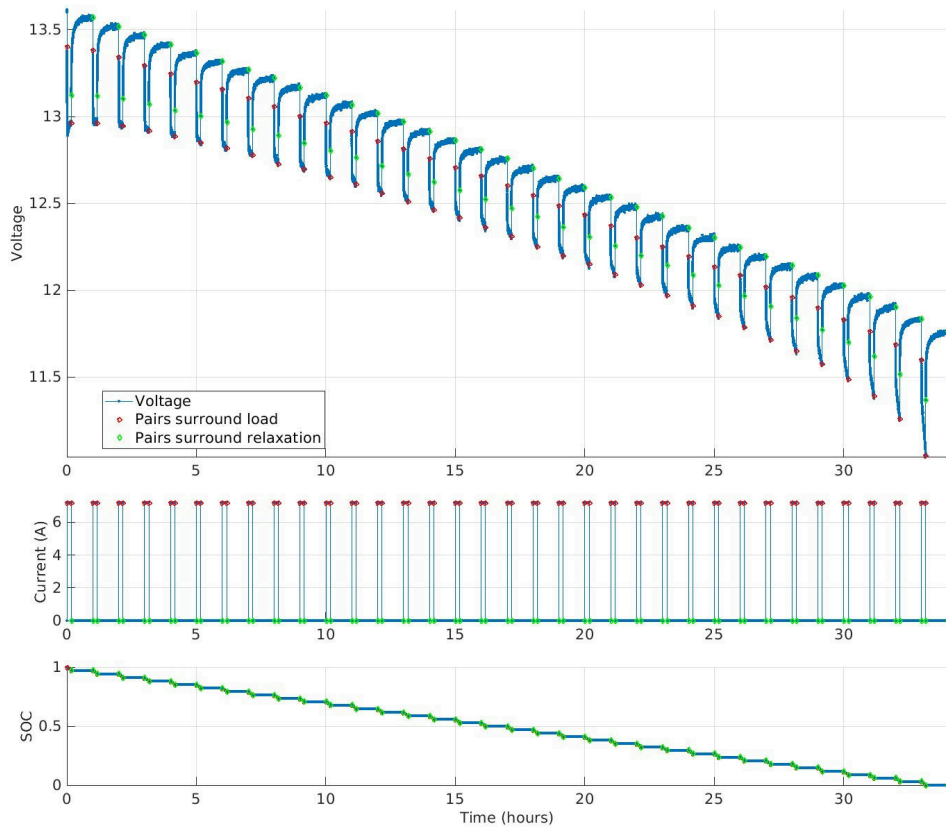


Abbildung 25: Identifizierte Pulse mittels Matlab-Funktion

Aus der Grafik geht hervor, dass 35 Pulse stattgefunden haben, bis die Batterie entladen war. Dazu lässt sich nun herausfinden, wie viele Amperestunden verbraucht wurden:

$$Q_{ges} = 7.2A \times \frac{1}{6}h \times 35 = 42Ah$$

Die Differenz zur Nennkapazität von 44 Ah lässt sich durch die Abweichung der C-Rate erklären. Die Nennkapazität der Batterie beträgt 22 Ah bei C20. Dies ergibt einen Entladestrom von 1.1 A pro Batterie. Die C-Rate des Tests beträgt rund C6.11 womit die Nennkapazität erwartungsgemäss tiefer ist. Zusätzlich ist das Alter der Batterie unbekannt, somit auch der genaue State of Health.

Mittels Optimierungs-Toolbox von Matlab können nun die relevanten Werte für den Innenwiderstand R_0 und die RC-Glieder (τ und R_x) sowie die Leerlaufspannung E_m berechnet werden [21]. Zu diesem Zweck versucht Matlab in jedem Puls, Werte zu finden, um das Verhalten möglichst präzise abzubilden. Abbildung 26 zeigt exemplarisch, wie der Spannungsverlauf nachgebildet wird.

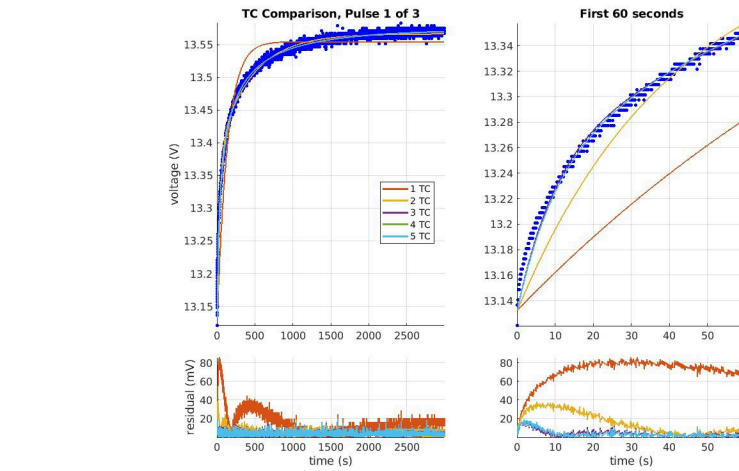


Abbildung 26: Optimierungsfunktion (curve fitting) von Matlab

Dieses *curve fitting* wird für jeden SOC-Schritt durchgeführt. Resultierend erhält man Tabellen für die Werte des ECM bei jedem SOC-Level. Abbildung 27 stellt die Werte anhand von Graphen dar.

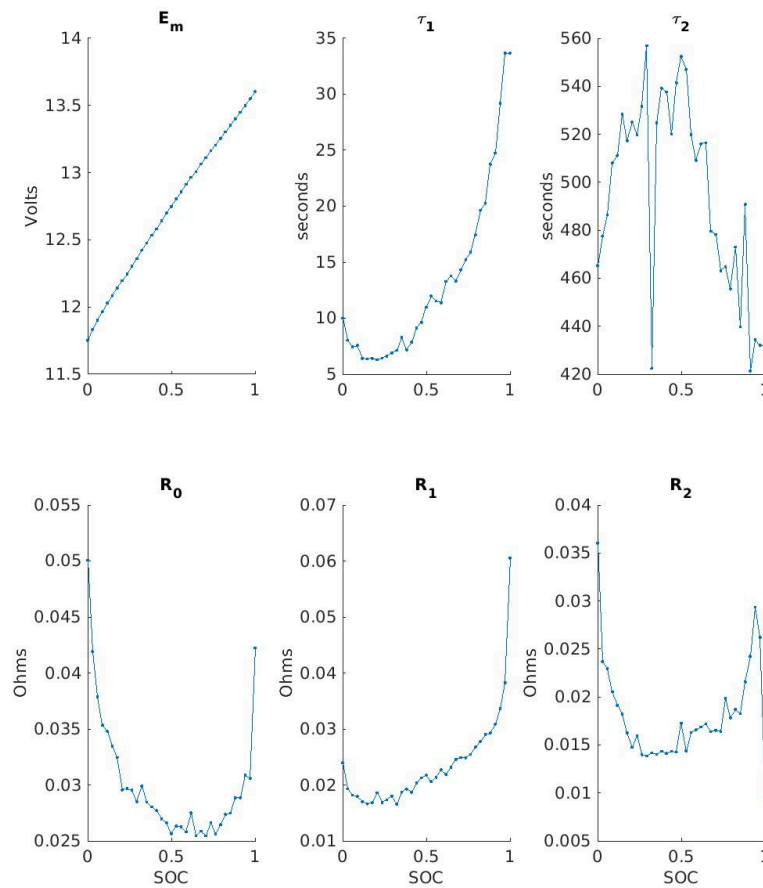


Abbildung 27: Identifizierte Parameter

Auffällig ist, dass die Leerlaufspannung E_m relativ linear ist. Dieser Verlauf ist bei anderen Batteriearten wesentlich nicht-linearer, wobei sich das OCV-Verfahren für die Bestimmung des SOC in Offline-Systemen eignen würde.

Die berechneten Werte finden nun im Kalman Filter in Form von Lookup-Tabellen Verwendung. Zu jedem SOC-Wert können die internen Parameter der Batterie abgerufen werden. Dies legt die Grundlage für die Implementierung des Filters.

3.1.2 Implementierung EKF

Die Umsetzung beinhaltet nebst den grundlegenden Berechnungen des EKF auch einige Spezialitäten zur Berechnung des SOC, die aufgezeigt werden sollen.

3.1.2.1 Predict

Die *predict*-Funktion des SOC-EKF beinhaltet die folgenden Gleichungen. Für die Extrapolation des States und dessen Kovarianz gilt:

$$\hat{x}_{n+1,n} = f(\hat{x}_{n,n}) + Gu_n$$

$$P_{n+1,n} = \frac{\partial f}{\partial x} P_{n,n} \left(\frac{\partial f}{\partial x} \right)^T + Q$$

Die systemspezifischen Formeln für die EKF-Implementierung werden durch die Matrizen $\mathbf{F}$ und $\mathbf{G}$ definiert. Für ein Equivalent Circuit mit zwei RC-Gliedern ergeben sich folgende zu implementierende Gleichungen.

Die State Transition Matrix und die Control Matrix ist wie folgt definiert:

$$\mathbf{F}_d = \frac{\partial(f)}{\partial(x)} = \begin{pmatrix} 1 & 0 & 0 \\ 0 & e^{\frac{-T_s}{\tau_1(\text{SOC}, T)}} & 0 \\ 0 & 0 & e^{\frac{-T_s}{\tau_2(\text{SOC}, T)}} \end{pmatrix}$$

$$\mathbf{G}_d = \begin{pmatrix} \frac{-T_s}{3600 \cdot \text{AH}} \\ \left(1 - e^{\frac{-T_s}{\tau_1(\text{SOC}, T)}} \right) \cdot R_1(\text{SOC}, T) \\ \left(1 - e^{\frac{-T_s}{\tau_2(\text{SOC}, T)}} \right) \cdot R_2(\text{SOC}, T) \end{pmatrix}^T$$

Dabei beinhaltet der State den SOC-Wert sowie die beiden Spannungen der RC-Glieder. Die Variable u ist in diesem System der gemessene Strom.

$$\mathbf{x} = \begin{pmatrix} \text{SOC} \\ V_{\text{RC1}} \\ V_{\text{RC2}} \end{pmatrix}$$

$$u = I_{\text{meas}}$$

In dieser Phase wird der nächste State aufgrund des letzten extrapoliert. Das heisst, es wird ein SOC-Wert und die Spannungen über den RC-Gliedern für den nächsten Zeitschritt abgeschätzt. Dies erfolgt in zwei Schritten. Zum einen wird der neue State ohne Stromeinfluss extrapoliert. Der SOC-Wert bleibt gleich, wobei jedoch die Spannungen über den RC-Gliedern abklingen. Der zweite Schritt extrapoliert den State anhand des fliessenden Stroms. Der SOC Wert ändert sich dabei anhand des Coulomb-Counting über einen Zeitschritt. Der Strom führt auch zu einem Anstieg der beiden RC-Glied Spannungen. Diese beiden Effekte zusammen ergeben die Schätzung für den nächsten Zeitschritt.

Folgend ist die Implementierung der `predict`-Funktion aufgeführt.

```

1  def predict(self, u: float, capacity_ah: float):
2      dt = self._dt # time step
3      x = self._x   # current state
4      P = self._P   # covariance matrix
5      Q = self._Q   # process noise matrix
6      soc = x[0]    # current soc
7
8      # lookup battery parameters for the given soc state
9      r1 = self._battery_params.lookup('r1', soc)
10     c1 = self._battery_params.lookup('c1', soc)
11     r2 = self._battery_params.lookup('r2', soc)
12     c2 = self._battery_params.lookup('c2', soc)
13     # calculate exponential terms
14     exp1 = np.exp(-dt / (r1 dot c1))
15     exp2 = np.exp(-dt / (r2 dot c2))
16     # state transition matrix
17     F = np.array([
18         [1.0, 0.0, 0.0],
19         [0.0, exp1, 0.0],
20         [0.0, 0.0, exp2]
21     ])
22     # control matrix (input transition matrix)
23     G = np.array([
24         -dt / (3600 dot capacity_ah),
25         r1 dot (1 - exp1),
26         r2 dot (1 - exp2)
27     ])
28     # extrapolate state
29     self._x = F @ x.T + G dot current
30     # extrapolate covariance
31     self._P = F @ P @ F.T + Q

```

Listing 1: Implementierung predict-Funktion SOC Kalman-Filter

Die Zeilen 1 - 12 sind vorbereitend und dienen hauptsächlich dazu, den Code verständlich zu machen. Erwähnenswert dabei ist, dass der Lookup (Zeilen 9 - 12) auf Basis des momentanen SOC beruht. Um die RC-Werte zu einem beliebigen SOC-Wert zu erhalten, werden die Werte aus den Lookup-Tabellen interpoliert.

Auf Zeile 17 und 23 werden vorbereitend die beiden Matrizen F und G definiert. Diese werden anschliessend in den Berechnungen auf Zeile 29 und 31 verwendet, um den nächsten State vorherzusagen.

3.1.2.2 Update

Für die *update*-Funktion werden die folgenden Gleichungen benötigt:

$$\begin{aligned}\hat{x}_{n,n} &= \hat{x}_{n,n-1} + K_n(z_n - h(\hat{x}_{n,n-1})) \\ \mathbf{P}_{n,n} &= \left(I - K_n \frac{\partial h}{\partial x}\right) P_{n,n-1} \times \left(I - K_n \frac{\partial h}{\partial x}\right)^T + K_n R_n K_n^T \\ \mathbf{K}_n &= P_{n,n-1} \left(\frac{\partial h}{\partial x}\right)^T \times \left(\frac{\partial h}{\partial x} P_{n,n-1} \left(\frac{\partial h}{\partial x}\right)^T + R_n\right)^{-1}\end{aligned}$$

Wobei die Jacobi Messmatrix $\mathbf{H}$ und die Messfunktion h von Bedeutung sind.

$$\mathbf{H}_d = \begin{pmatrix} \frac{\partial V_{OC}}{\partial SOC} & -1 \end{pmatrix}$$

$$h(x, i) = OCV(SOC) - I \cdot R_0 - U_{RC1} - U_{RC2}$$

Anhand des aktuellen SOC-Wert wird in den Lookup-Tables nachgeschaut, welche Werte für die Leerlaufspannung (OCV) und den Innenwiderstand (R_0) zu verwenden sind. Die geschätzte Spannung setzt sich nun aus Leerlaufspannung abzüglich des Spannungsabfalls über dem Innenwiderstand und den beiden Spannungen über den RC-Glidern zusammen. Die Ableitung für die Messmatrix $\mathbf{H}$ ist vorberechnet und stammt auf Zeile 10 ebenfalls aus den Lookup-Tabellen. Der Wert beschreibt, wie stark sich die OCV bei gegebenem SOC verändert.

Folgend ist die Implementierung der **update**-Funktion aufgeführt:

```

1  def update(self, z):
2      # measurements
3      u_meas, i_meas = z # measured voltage and current
4      soc = self._x[0] # current soc
5      u_rc1 = self._x[1] # rc1 voltage
6      u_rc2 = self._x[2] # rc2 voltage
7      # lookup battery parameters
8      ocv = self._battery_params.lookup('ocv', soc)
9      r0 = self._battery_params.lookup('r0', soc)
10     dv0_dsoc = self._battery_params.lookup('dv0', soc)
11     # Update measurement matrix H with current OCV derivative
12     self._H[0, 0] = dv0_dsoc
13     H = self._H # measurement matrix
14     P = self._P # state covariance matrix
15     R = self._R # measurement noise
16     # voltage loss caused by the battery's internal resistance
17     u_intern = r0 * i_meas
18     # predicted terminal voltage
19     u_pred = ocv - (u_intern + u_rc1 + u_rc2)
20     # measurement residual (innovation)
21     y = u_meas - u_pred
22     # innovation covariance
23     S = H @ P @ H.T + R
24     # kalman gain
25     K = P @ H.T @ np.linalg.inv(S)
26     # update state estimate
27     self._x = self._x + K.flatten() * y
28     # update Covariance
29     self._P = (np.eye(3) - K @ H) @ P
30     # ensure SOC stays within bounds
31     self._x[0] = np.clip(self._x[0], 0, 1)

```

Listing 2: Implementierung update-Funktion SOC Kalman-Filter

3.1.2.3 Verifizierung EKF

Um die Implementierung zu testen, wurden die Simulationsdaten aus Matlab exportiert. Mit einem Simulationsscript kann nun verglichen werden, wie sich die Implementation zu den Simulationsdaten mit gleichen Inputs verhält.

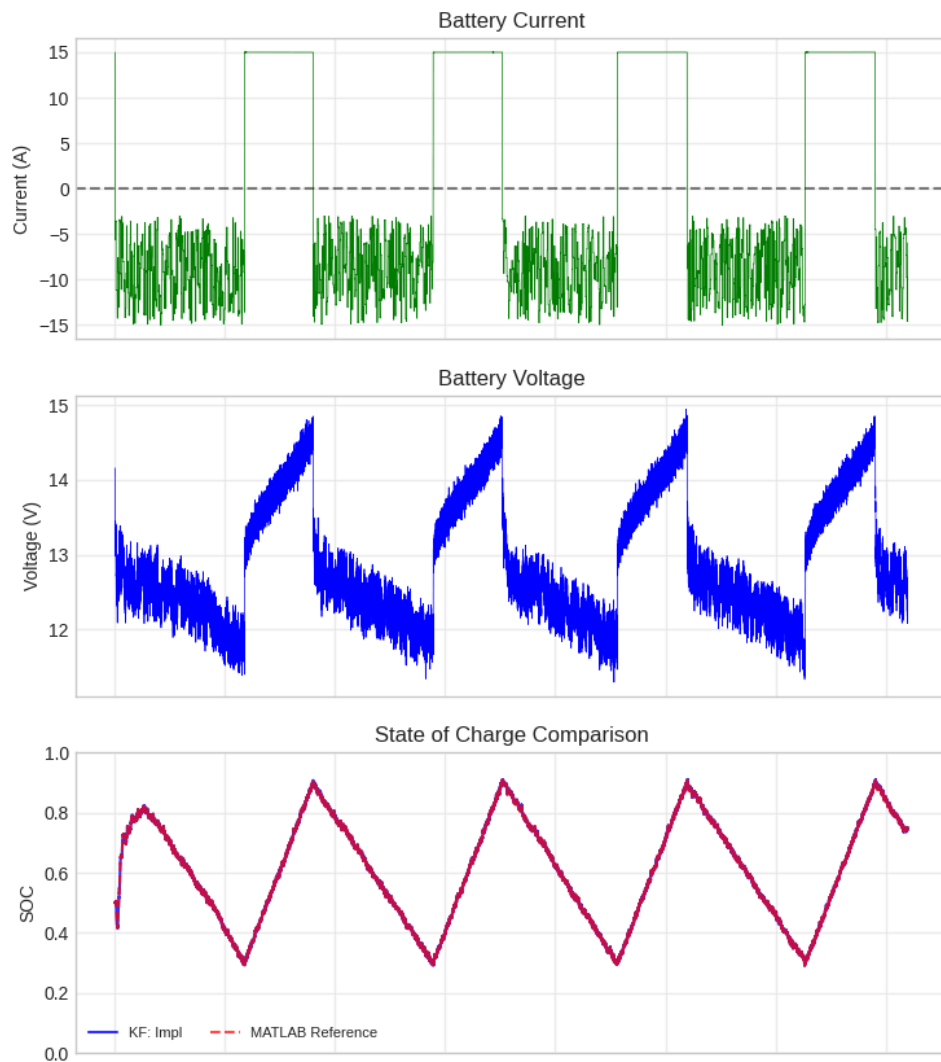


Abbildung 28: Vergleich Matlab-Simulation vs. EKF-Implementation

Die Abbildung 28 visualisiert die Daten über mehrere Lade- und Entladezyklen. Dabei ist zu sehen, dass die Werte der Implementierung und der Simulation aus Matlab überlappend sind.



Abbildung 29: Detailvergleich Matlab-Simulation vs. EKF-Implementation

Auf dem Detailvergleich auf Abbildung 29 sind jedoch leichte Abweichungen zu erkennen. Möglicherweise lassen sich diese durch unterschiedliche Datentypen oder Linearisierungsfunktion der Lookup-Tabellen erklären. Da die Abweichungen eher gering sind, wird von einer korrekten Implementierung ausgegangen.

3.2 SOH - Extended Kalman Filter

Um den sicheren Betrieb zu gewährleisten, ist es wichtig zu wissen, wann eine Batterie ihre Leistung nicht mehr erbringen kann. Dieser Wert beruht auf dem Resultat des bereits implementierten EKF für SOC. Die Schätzung des SOH ist ebenfalls ein EKF, welcher die aktuell verfügbare Kapazität der Batterie schätzt. Der SOH wird anhand der Kapazitäts-Schätzung, der Nennkapazität und der End-of-Life Kapazität berechnet. Der geschätzte Wert für die momentane Kapazität wird wiederum als Input für die SOC-Schätzung verwendet.

Der EKF basiert auf dem Baustein von Matlab Simulink *SOH Estimation (capacity based)* [23]. Die Inputs für den EKF sind Strom, die geschätzte Kapazität des SOC-EKF sowie die initiale Kapazität der Batterie.

3.2.1 Systemmodell

Prinzipiell vergleicht der Filter die Differenz des SOC mit dem gemessenen Stromfluss über die Zeit (Coulomb Counting). Daraus ergibt sich folgendes Systemmodell:

$$f(Q) = 0$$

Die Erwartung ist keine Veränderung der Kapazität während eines Zeitschrittes.

Für die Messfunktion h gilt daher:

$$0 = \frac{\partial(\text{SOC})}{\partial(t)} + \frac{i}{3600 \cdot Q}$$

Daraus lassen sich die Jacobians berechnen:

$$\mathbf{F}_d = 1$$

$$\mathbf{H}_d = -\frac{i \cdot T_s}{3600 \cdot Q^2}$$

3.2.2 Implementierung EKF

Die *predict*-Funktion ist trivial und extrapoliert denselben State lediglich weiter. Auch die Unsicherheit wird nur durch die Variable Q erhöht.

```
1 def predict(self):  
2     # extrapolate the state  
3     self._x = self._x  
4     # extrapolate uncertainty  
5     self._P = self._P + self._Q
```

py

In der *update*-Funktion findet die etwas aufwändigere Berechnung der Messgleichungen statt. Zunächst wird auf Zeile 17 die erwartete Kapazität berechnet, die anschliessend zur Berechnung des Residuals auf Zeile 23 Verwendung findet. Anhand des Kalman Gain wird dann eine neue Kapazitäts-Schätzung gemacht. Mittels der Matrix $\mathbf{H}$ wird der Kalman Gain berechnet und die Unsicherheit aktualisiert.

```

1  def update(self, z):
2      current, soc = z
3      # first update, no derivation available
4      if self._prev_soc is None:
5          self._prev_soc = soc
6          return self._x
7      P = self._P
8      x = self._x
9      soc_dot = (soc - self._prev_soc) / self._dt
10     self._prev_soc = soc
11     h = soc_dot + current / (3600.0 * x)
12     H = -current / (3600.0 * x**2)
13     # compute the kalman gain
14     S = H * P * H + self._R
15     K = P * H / S
16     # update estimate with measurement
17     self._x = x + K * (0.0 - h)
18     # update the estimate uncertainty
19     self._P = (1 - K * H) * P
20     self._x = np.clip(self._x, 0, self._Q_init)
21     return self._x

```

3.2.2.1 Verifizierung EKF

Zur Verifizierung des SOH-EKF wurden Simulationen in Matlab durchgeführt und dessen Daten exportiert. Mit denselben Input-Daten wurde in diesem Fall auch das exakt selbe Resultat erreicht. Wie in Abbildung 30 zu sehen ist, verlaufen die beiden Linien übereinander.

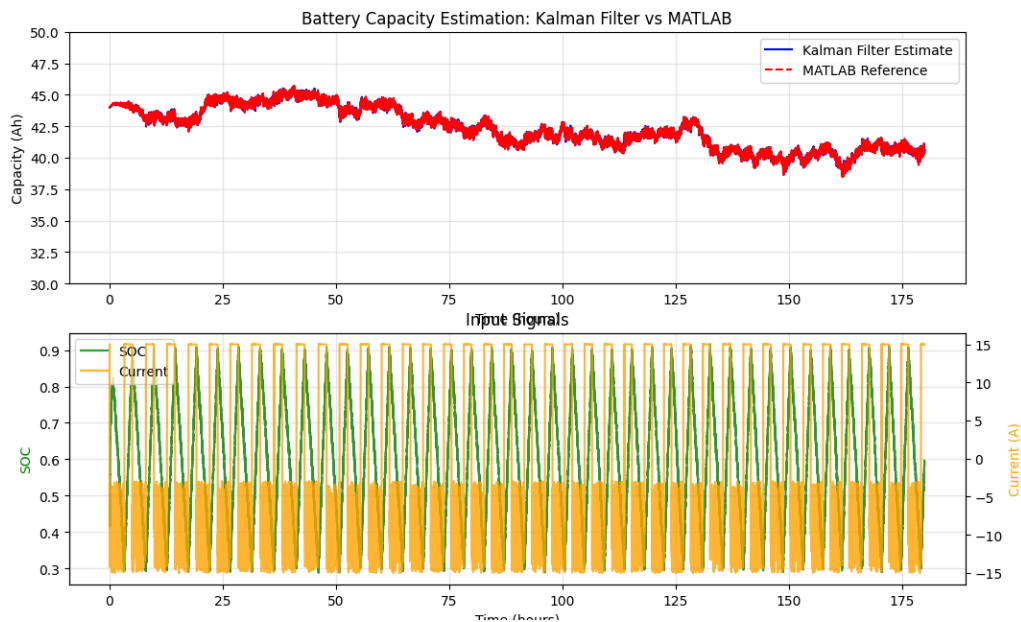


Abbildung 30: Equivalent Circuit Model (Quelle: [17])

Der Test beinhaltet 40 Zyklen über 175 Stunden. Der Kapazitätsverlust der Batterie ist dabei in Matlab manuell definiert.

3.3 Roboterintegration

Dieses Kapitel handelt von der Integration entwickelter Algorithmen in das bestehende Roboter-System. Dabei wird aufgezeigt, wie die Daten für die beiden Kalman Filter bereitgestellt werden und diese mit dem System interagieren.

3.3.1 Datenerfassung

Für die SOC- und SOH-Schätzungen sind Strom- und Spannungsdaten der Batterie notwendig. Zu diesem Zweck wurde ein preiswerter Sensor von Peacefair (PZEM 017-3) [24] verwendet. Über eine RS485-Schnittstelle lassen sich die Messwerte bis maximal zweimal pro Sekunde auslesen. Die Strommessung erfolgt dabei über einen Messwiderstand, auch Shunt genannt. Ein Shunt ist ein kleiner temperaturunabhängiger Widerstand. Durch den Spannungsabfall über dem Shunt kann auf den fließenden Strom zurückgeschlossen werden.

Die dazugehörige Formel lautet:

$$I = U \cdot R_{\text{Shunt}}$$

Die folgende Abbildung 31 zeigt die Systemarchitektur der eingesetzten Komponenten.

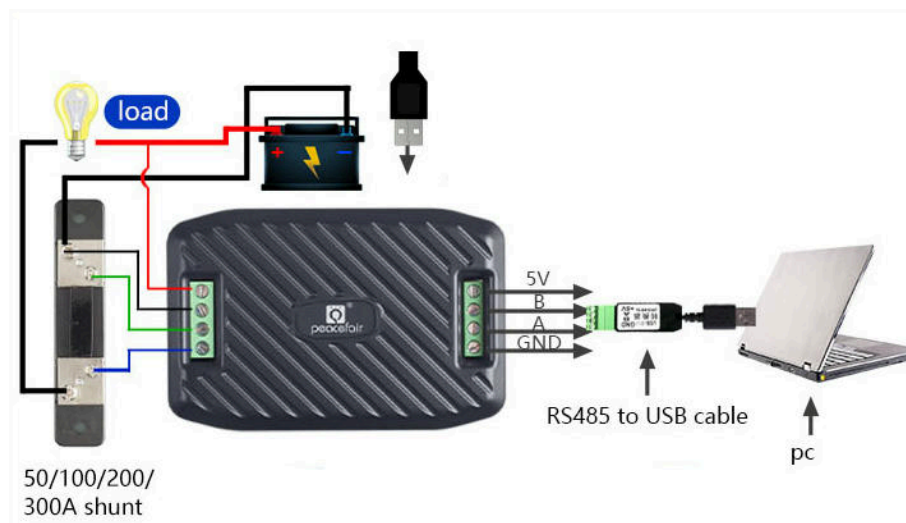


Abbildung 31: Peacefair PZEM 017-3

Die Glühlampe, welche mit *load* beschriftet ist, repräsentiert in diesem Fall die Roboter-Plattform. Die ganze Leistung für den Roboter fließt über diesen Shunt, welcher sich direkt vor der Batterie befindet. So lassen sich die effektiven Batteriewerte aufnehmen. Der Sensor wird direkt von der Batterie über die rote und schwarze Verbindung gespeist. Die Daten gelangen über die USB-Schnittstelle an den in der Roboter-Plattform verbauten ITX-Mini-Computer.

3.3.2 ROS Integration

Für die Integration der SOC- und SOH-Schätzung sind im Wesentlichen drei ROS2-Nodes und drei Topics involviert. Die Ausgangslage stellt eine von der Roboter-Plattform versendete *battery_status* Nachricht dar, die auf dem Topic `{ns}/platform/bms/state` verfügbar ist. Dieser

Nachrichtentyp enthält alle relevanten Batterie-Informationen, welche für den Betrieb von Interesse sind.

Das folgende Listing 3 zeigt eine Beispielnachricht des Typs *Battery_State*.

```

1  {
2    "header": {
3      "stamp": {
4        "sec": 1756384522,
5        "nsec": 55211134
6      },
7      "frame_id": "battery_frame"
8    },
9    "voltage": 12.619999885559082,
10   "temperature": null,
11   "current": 5.789999961853027,
12   "charge": 19.41096305847168,
13   "capacity": 44,
14   "design_capacity": 44,
15   "percentage": 44.11582565307617,
16   "power_supply_status": 2,
17   "power_supply_health": 1,
18   "power_supply_technology": 0,
19   "present": true,
20   "cell_voltage": [],
21   "cell_temperature": [],
22   "location": "",
23   "serial_number": ""
24 }

```

Listing 3: ROS2 Battery State Nachricht (Quelle: [25])

Die von der Plattform veröffentlichten Daten sind jedoch teilweise unbrauchbar. Strom- und Spannungswerte werden nicht direkt an der Batterie aufgenommen, wodurch eine Abweichung zu den vom Sensor gemessenen Daten festzustellen ist. Die SOC-Werte sind von der Batteriespannung abgeleitet. Dadurch entstehen unpräzise Werte, insbesondere bei wechselnder Belastung.

Um dies zu korrigieren, sind die folgenden Nodes und Topics involviert. Die folgende Abbildung 32 zeigt eine Übersicht der Kommunikationswege.

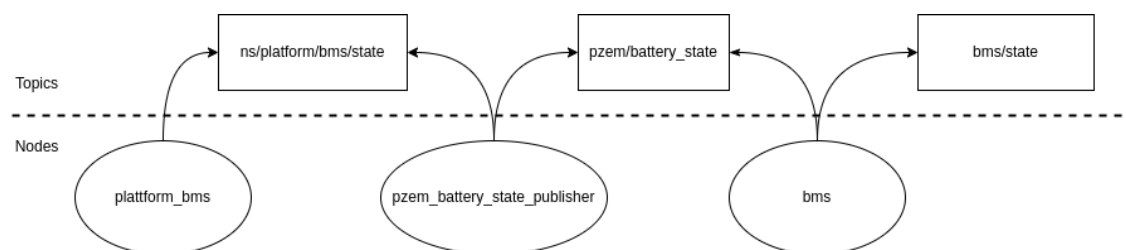


Abbildung 32: Overview ROS2 Nodes

In einem ersten Schritt werden die Strom und Spannungsdaten ersetzt. Dazu versendet der Node *pzem_battery_state_publisher* die ursprüngliche Nachricht mit Strom- und Spannungswerten des Sensors. Diese Nachricht wird anschliessend auf dem Topic *pzem/battery_state* veröffentlicht. Der Node *bms* implementiert die SOC- sowie SOH-Schätzung. Dazu abonniert er die *battery_state*

Nachricht auf dem *pzem/battery_state* Topic. Anhand dieser Daten werden nun die beiden Schätzungen berechnet. Erneut werden Werte aus der *battery_state* Nachricht durch die Resultate der Filter ersetzt und auf dem Topic *bms/state* mit einer Frequenz von 1 Hz publiziert [26].

3.4 Energieverbrauch

3.4.1 Analyse Stromverbrauch

Anhand einer Aufzeichnung des Stromverbrauchs während eines Bootups lässt sich ein Energieprofil erstellen. Folgende Abbildung 33 visualisiert die Energiedaten.

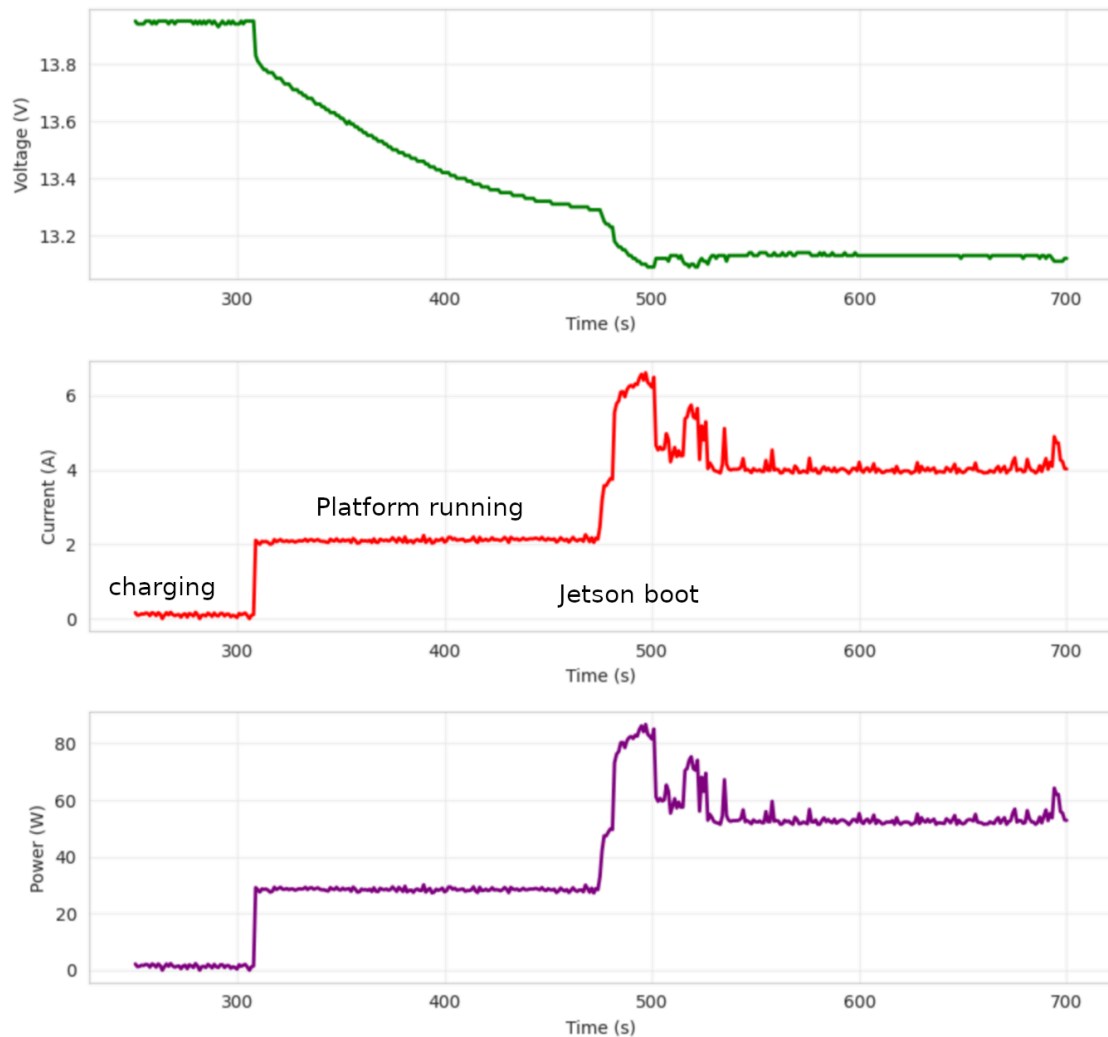


Abbildung 33: Energieverbrauch bei Roboter-Start

Zu Beginn befindet sich der Roboter mit geladener Batterie am Ladegerät, wodurch nahezu kein Strom fließt. Nach Sekunde 300 wird das Ladegerät entfernt und der Strom steigt auf den Grundverbrauch der Plattform. Der Energieverbrauch liegt bei rund 30 Watt. Der nächste Stromanstieg markiert den Boot-Prozess des Jetson-Computer. Anschliessend stabilisiert sich der Energieverbrauch bei 55 Watt. Während des Betriebs ist auf dem Jetson eine höhere Belastung zu erwarten. Der Gesamtverbrauch kann dabei bis zu 60 Watt betragen.

Die folgende Abbildung 34 zeigt den Energieverbrauch während einer Fahrt.

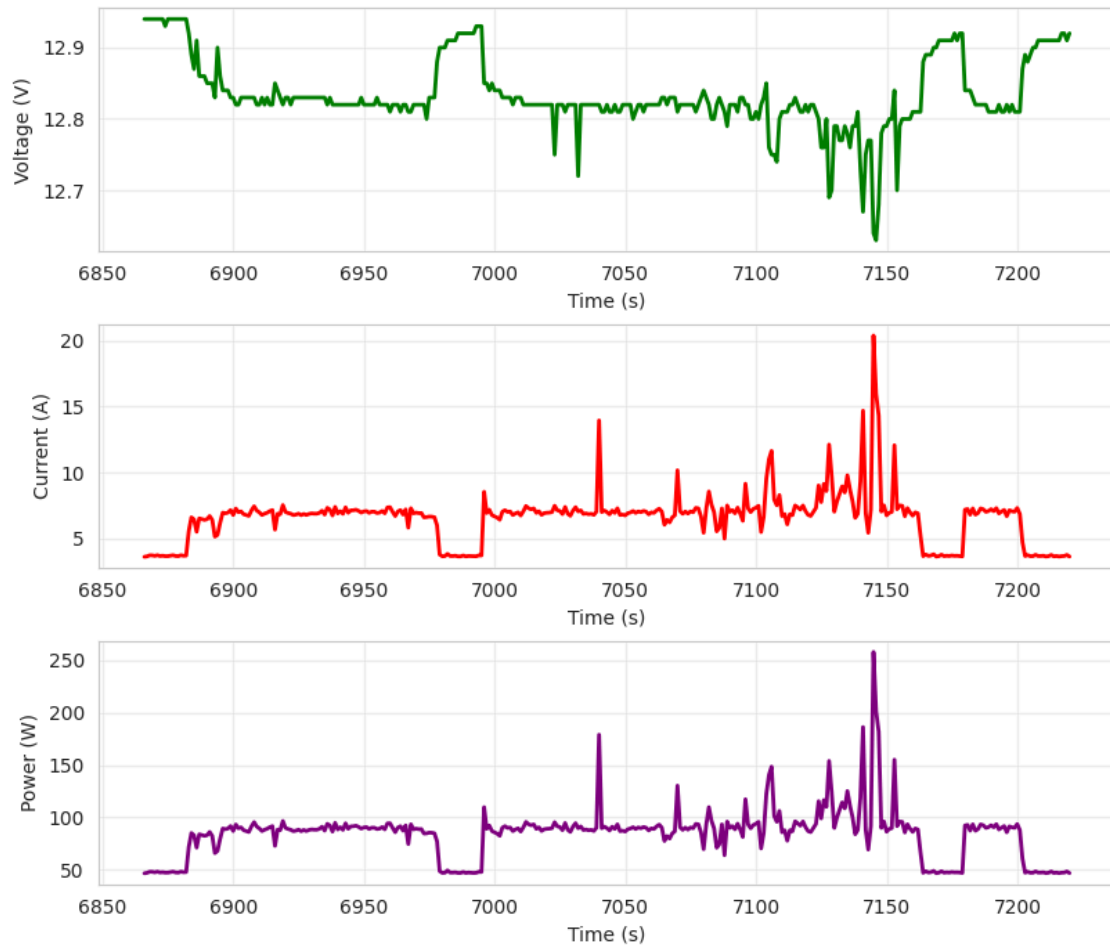
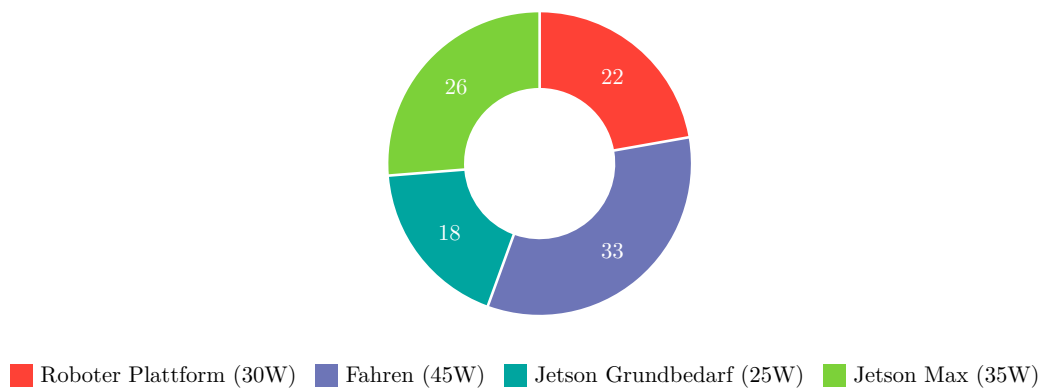


Abbildung 34: Energieverbrauch bei Fahrt

Die ruhigen Phasen mit wenig Ausreisser repräsentieren normales Fahrtempo. Die deutlich unruhigere Phase bei Sekunde 7100 wurde während schneller Fahrt aufgenommen. Der grösste Ausreisser bei Sekunde 7150 ist Fahrt mit leichtem Widerstand. In realen Situationen ist von einem normalen Fahrtempo auszugehen, wobei der Energieverbrauch bei run 100 Watt liegt. Abzüglich des Grundverbrauchs durch die anderen Systeme ergibt dies einen Energieaufwand von 45 Watt.

Auf Basis dieser Analyse ergibt sich folgende Energiezusammensetzung für das komplette System bei Fahrt:



Diese Daten sind als Richtwerte zu verstehen und keine präzisen Messungen. Je nach Systembelastung, Geschwindigkeit und weiteren Faktoren können die Werte abweichen. Unter der Annahme, dass der Jetson durch ML-Tasks auf der GPU voll ausgelastet ist, beträgt die Gesamtleistung rund 135 Watt.

3.4.2 Reichweitenabschätzung

Aus den SOC- und SOH-Daten sowie dem Energiebedarf lässt sich nun berechnen, wie lange der Roboter betrieben oder welche Distanz zurückgelegt werden kann. Die verfügbare Energie des Roboters kann wie folgt berechnet werden [27]:

$$E_{\text{rem}} = U_{\text{nom}} \cdot \text{SOC} \cdot Q_{\text{nom}}$$

- E_{rem} - Restliche Energie im Akkumulator (Wh)
- U_{nom} - nominelle Batterie Spannung (V)
- SOC - State of Charge
- Q - nominelle Kapazität der Batterie (Ah)

Bei neuer und geladener Batterie (SOC = 1, SOH = 1) ergibt sich somit eine Laufzeit von:

$$t_{\text{rem}} = \frac{E_{\text{rem}}}{E_{\text{avg}}} = \frac{12\text{V} \cdot 44\text{Ah}}{135\text{W}} = 3.91\text{h}$$

Da der Roboter im Pflegeheim Personen begleitet, wird von einer langsamen Schrittgeschwindigkeit von 3 km/h ausgegangen. Diese entspricht auch ungefähr der normalen Fahrt der Energiemessung. Die Reichweite lässt sich wie folgt berechnen:

$$s = v_{\text{avg}} \cdot t_{\text{rem}}$$

Für neue und geladene Batterie ergibt das:

$$s = v_{\text{avg}} \cdot t_{\text{rem}} = 3\text{km/h} \cdot 3.91\text{h} = 11.73\text{km}$$

3.4.3 Ladestrategie

3.4.3.1 Ausgangslage

Im Projekt WSR soll der Roboter autonom Bewohner zu Therapien begleiten. Voraussetzung dafür ist, dass die bevorstehenden Termine bekannt sind. Somit sind Auftragslage und Auftragsvolumen kalkulierbar. Des Weiteren ist im folgenden davon auszugehen, dass sich der Roboter selbständig an einem Dock aufladen kann.

3.4.3.2 Prioritäten

Höchste Priorität hat jeweils das Erledigen der Aufgaben. Durch Pausen und Wartezeiten entstehen weitere optimierbare Aspekte. Im Folgenden werden drei Varianten aufgezeigt.

- **Batterielebensdauer:** Ein entscheidender Faktor für die Lebensdauer einer Blei-Säure Batterie ist die Tiefe der Entladung. Je weniger tief die Batterie entladen wird, desto mehr Zyklen können gemacht werden. Somit ist das Laden während Einsatzpausen vorteilhaft.

- **Informationsgenerierend:** Da der Roboter Heim-Bewohner vor dem Begleiten zu Terminen finden muss, ist die Datenbasis entscheidend. Einsatzpausen können dazu verwendet werden, Informationen zu sammeln. Dadurch entsteht ein kompletteres Bewegungsprofil, was die Suche erleichtert.
- **Stand-By:** Aus der Zusammensetzung des Energiebedarfs ist ersichtlich, dass ein Grossteil des Energiebedarfs nicht auf das Grundsystem zurückzuführen ist. Der Energieverbrauch könnte in Einsatzpausen auf ein Minimum reduziert werden, um Energie zu sparen und weniger häufig zu laden.

Nebst den Strategien spielt auch die Position des Lade-Docks eine entscheidende Rolle. Ein zentraler Ort und somit kurze Wege zum Dock erleichtern das häufige Laden der Batterie ohne viel Energieverlust für Hin- und Rückweg. Für eine abschliessende Empfehlung fehlen an dieser Stelle die Erfahrungen und Praxistests.

3.4.3.3 Ladevorgang

Wie in Abschnitt 2.3 erwähnt, wird die Ladestrategie von der Roboter-Plattform übernommen. Dazu erfolgt die Ladung in drei Schritten [5]:

1. Laden bei konstantem Strom von 6 A
2. Laden bei konstanter Spannung mit abnehmendem Strom von 6 A bis 2 A
3. Float Charging bei 14.6 V

Das Ladegerät des Herstellers ist genau für diesen Ladevorgang ausgelegt und kann maximal 6 A bei 14.6 V bereitstellen. Wird während des Ladevorgangs Leistung für den Betrieb verwendet, so verlängert dies die Ladezeit. Die folgende Abbildung 36 zeigt den Strom- und Spannungsverlauf, während geladen wird.

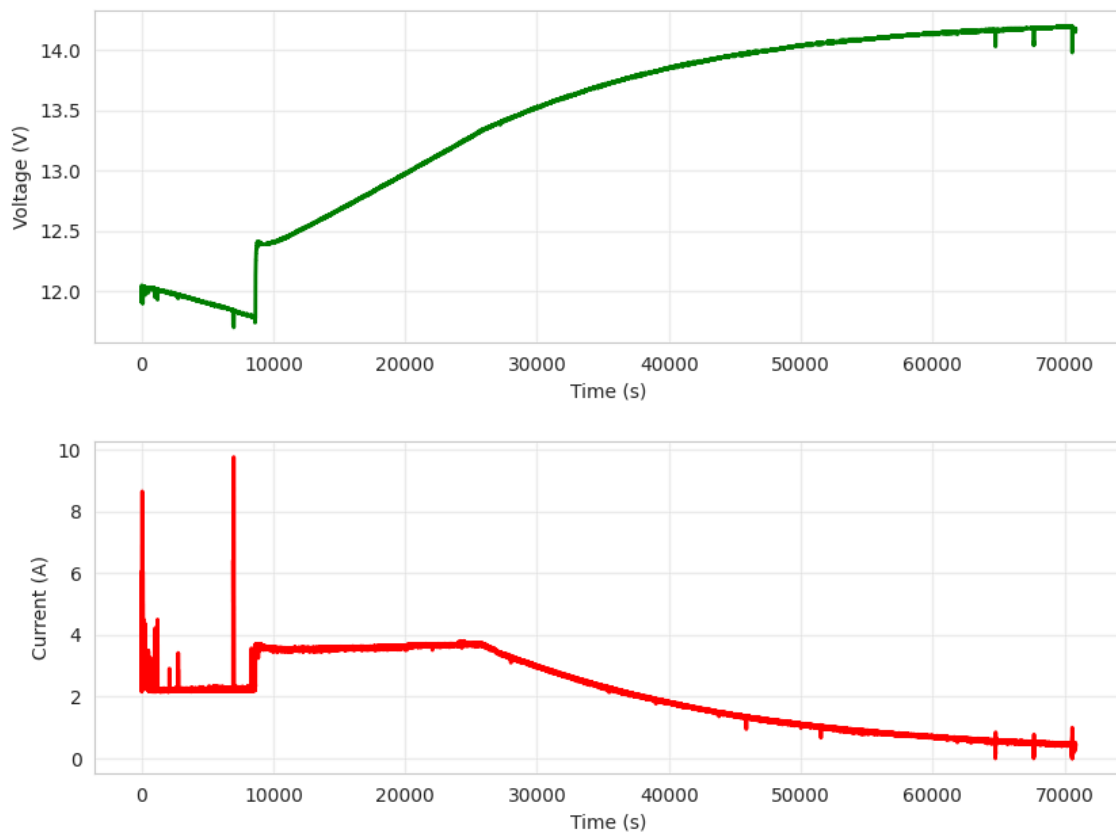


Abbildung 36: Batterie Ladekurve

Die Aufzeichnung beginnt während einer Phase der Entladung, somit sind die ersten rund 10k Sekunden nicht dem Ladevorgang zuzuweisen. Mit Beginn der Ladung steigen beide Werte. Hier ist zu erwähnen, dass die Polarität im Stromsignal nicht abgebildet ist. Der Ladestrom in der ersten Ladephase beträgt rund 3.5 A. Die Differenz zum erwarteten Ladestrom von 6 A ist durch die 2.5 A Betriebsstrom zu erklären. Ersichtlich ist zudem der Übergang in die nächste Ladephase, welche durch den sinkenden Strom erkennbar ist.

3.5 Monitoring

Die Informationen des Batterie Management System sollen für Benutzer und Entwickler zugänglich sein. Dazu werden relevante Daten mittels ROS-Node geloggt, so dass diese in einem Fehlerfall konsultierbar sind. Zudem wurde ein einfaches User Interface umgesetzt, um Einblick in die aktuellen Informationen zu gewähren. Die Abbildung 37 zeigt ein Screenshot des Interfaces.

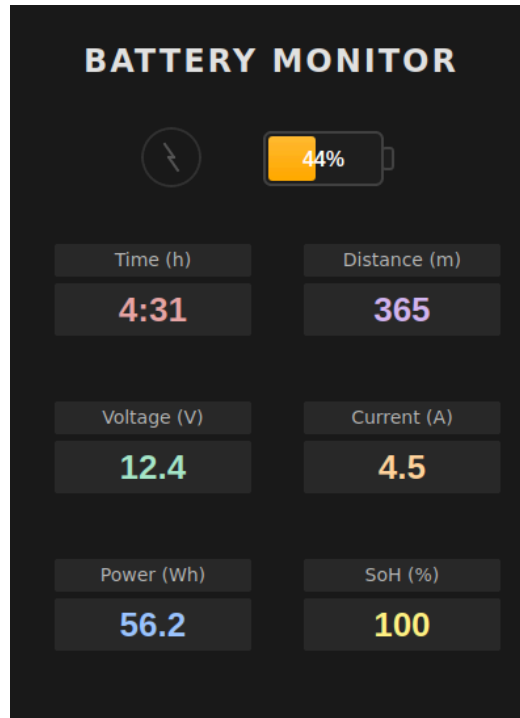


Abbildung 37: Battery Monitor GUI

4 Validierung

In diesem Kapitel wird untersucht, wie gut das entwickelte Batteriemanagementsystem (BMS) in der Realität funktioniert. Der Fokus liegt dabei auf den Schätzungen des SOC und SOH.

4.1 State of Charge

Die Batteriekapazität wurde vor dieser Arbeit von der Spannung abgeleitet. Dies ist ungenau, besonders bei wechselnder Strombelastung. Die entwickelte Lösung mittels Kalman Filter erzielt deutlich bessere Resultate. Im Folgenden werden einige Tests vorgestellt, die zur Validierung der SOC-Schätzung gemacht wurden.

4.1.1 Funktionstest

Für einen Funktionstest wurden SOC, Strom und Spannung über einen langen Zeitraum aufgezeichnet. Der Roboter war während dieser Zeit im Stillstand. Die Abbildung 38 zeigt die Visualisierung der Daten.

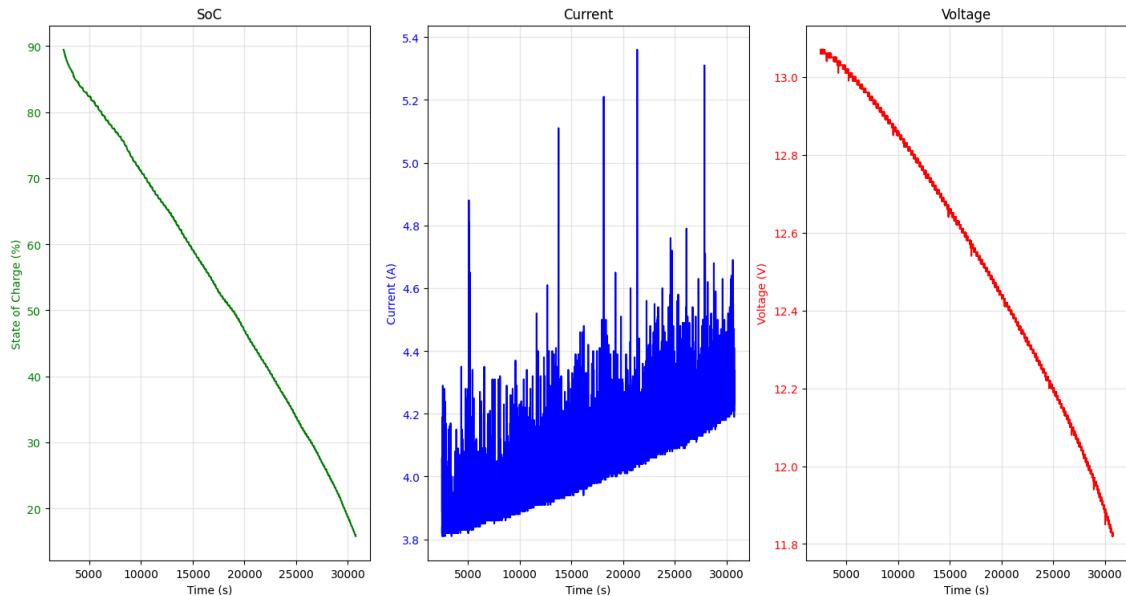


Abbildung 38: Funktionstest im Ruhezustand

Auffällig ist der steigende Strom über die Zeit. Dies hängt mit der sinkenden Spannung zusammen. Da das System mit einer konstanten Energie versorgt wird, ist folgende Gleichung erfüllt:

$$P = U \cdot I$$

Der SOC-Wert sieht relativ linear aus. Dieser startet bei 89,46% und endet bei 15,78%, womit 73,68% verbraucht wurden. Die beiden Batterien haben zusammen eine Nennkapazität von 44 Ah bei einer C-Rate von C20, also 2,2 A Belastung. Die zu erwartende Batteriekapazität dürfte also etwas geringer sein, aufgrund der etwas höheren Belastung und des unbekannten Alters. Der integrierte Strom über die Zeit beträgt 31,547 Ah. Daraus lässt sich die Gesamtkapazität der Batterie berechnen:

$$73,68\% \times C_{\text{ges}} = 31,54\text{Ah}$$

$$C_{\text{ges}} = \frac{31.54\text{Ah}}{0.7368}$$

$$C_{\text{ges}} = 42.82\text{Ah}$$

Das Resultat kann mit den vorhandenen Mitteln nicht verifiziert werden. Dafür braucht es entsprechende Laboreinrichtungen. Jedoch ist der Wert absolut legitim. Die zu erwartende Kapazität liegt in dieser Größenordnung.

4.1.2 Praktische Anwendung

Aus den BMS-Aufzeichnungen des Roboters geht hervor, dass das Strom- und Spannungsprofil in realistischen Szenarien deutlich anders aussieht, als bei der Simulation zu Beginn. Daher wurde in Matlab ein Stromprofil nachgestellt, welches die tatsächlichen Bedingungen besser abbildet. Die Abbildung 39 zeigt die Resultate der Simulation.

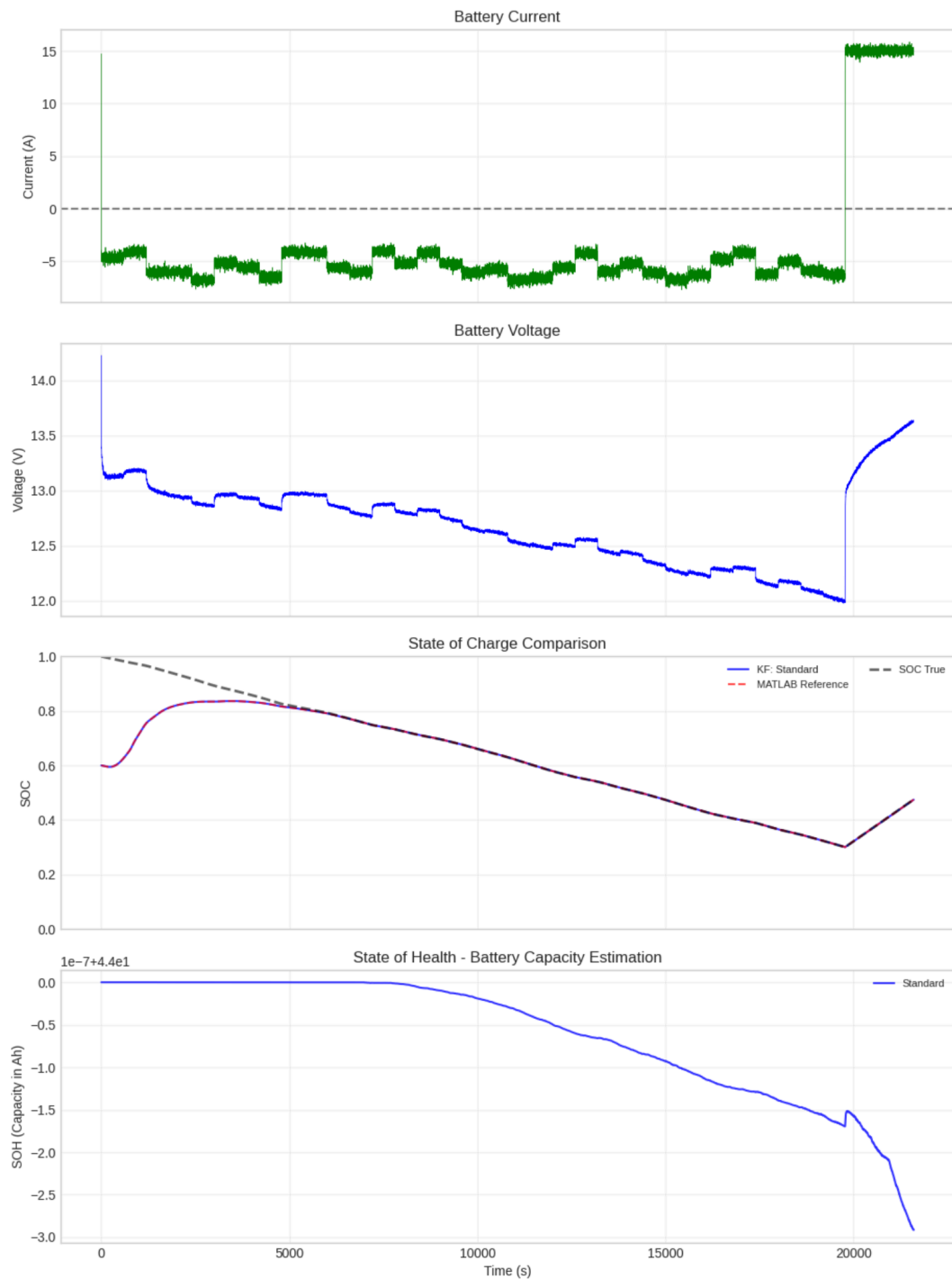


Abbildung 39: Vergleich SOC-Schätzung bei realistischem Stromverbrauch

Strom- und Spannungsverlauf weisen weniger Noise auf. Besonders das Spannungssignal hat sich als sauber herausgestellt. Das Resultat des SOC-Kalman Filter ist dabei identisch mit der Simulation in Matlab.

Um die SOC-Schätzung in einem praktischen Szenario zu untersuchen, wurden die BMS-Daten über einen Zeitraum von 7 Stunden aufgenommen. Die folgende Abbildung 40 visualisiert die interessanten Werte.

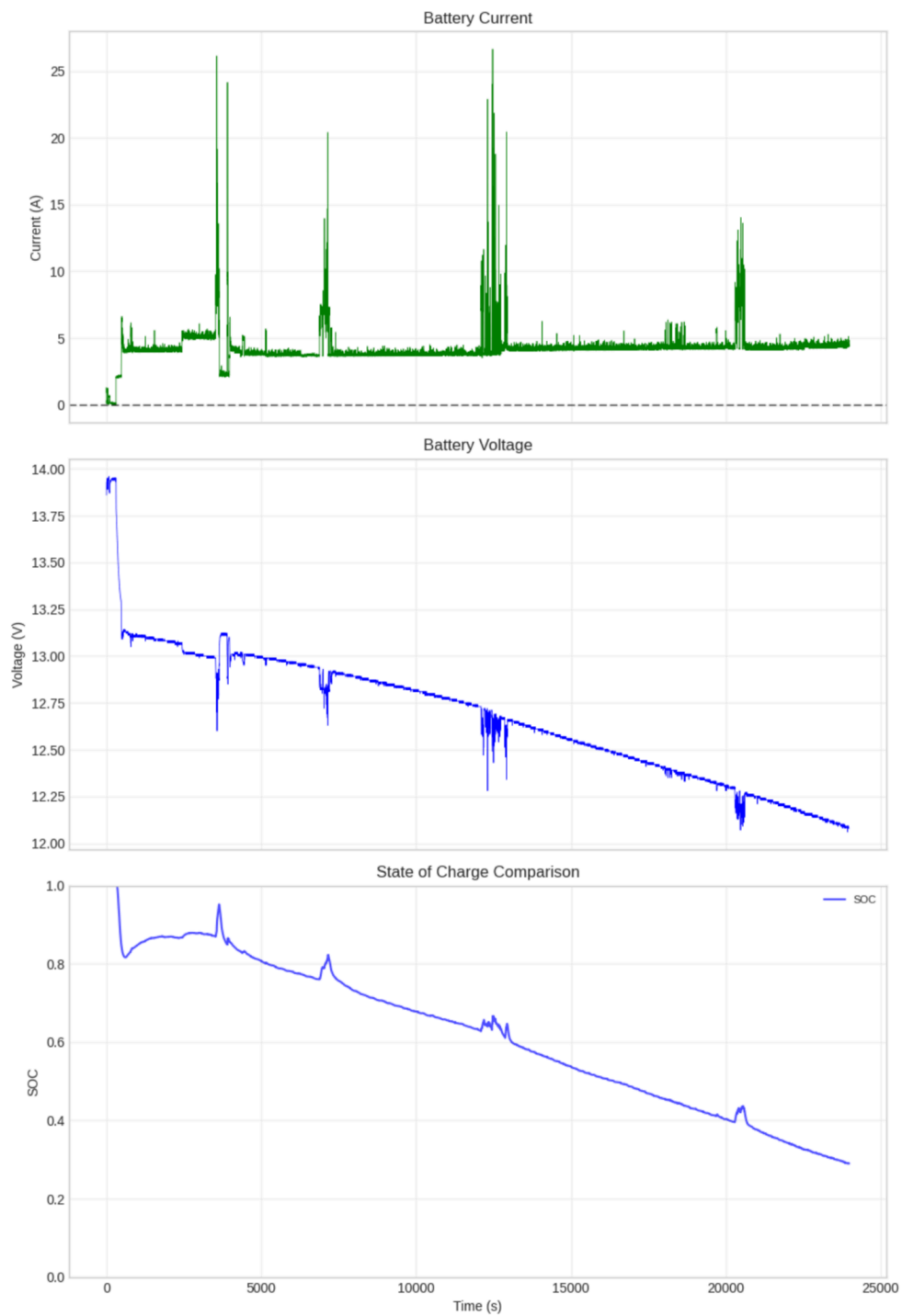


Abbildung 40: Langzeitaufnahme BMS-Daten

Während der ersten Phase befindet sich der Roboter am Ladegerät, wodurch die hohe Spannung und der tiefe Strom zu erklären sind. Nach Entfernen des Ladegeräts erfolgen über die gesamte Zeitspanne immer wieder Fahrten, welche durch den erhöhten Stromverbrauch erkennbar sind. Es fällt auf, dass die Ergebnisse des SOC-Kalman Filter unbefriedigend sind. Der Einbruch des SOC bei der Entfernung des Ladegeräts sowie die Anstiege nach Belastungen sind eindeutig falsche Schätzungen. Der Kalman Filter ist zu diesem Zeitpunkt mit den Defaultwerten aus dem

Matlab-Baustein konfiguriert. Dabei scheint es, als wäre die Gewichtung auf den Messdaten zu hoch. Basierend auf den gleichen Daten sind weitere Einstellungen getestet worden. Dabei muss ein Kompromiss zwischen Glättung und Geschwindigkeit der Konvergenz zum wahren Wert getroffen werden. Zu diesem Zweck wurden Versuche mit unterschiedliche Initialisierungen durchgeführt, sowie unterschiedlichen Werte für $\mathbf{Q}$ und $\mathbf{R}$. Die bisher verwendete Schätzung mit den Standardwerten ist in blauer Farbe dargestellt.

Die folgende Abbildung 41 zeigt durch Erhöhung von $\mathbf{R}$ und einer verkleinerten $\mathbf{Q}$ -Matrix deutliche Verbesserungen in der Glättung.

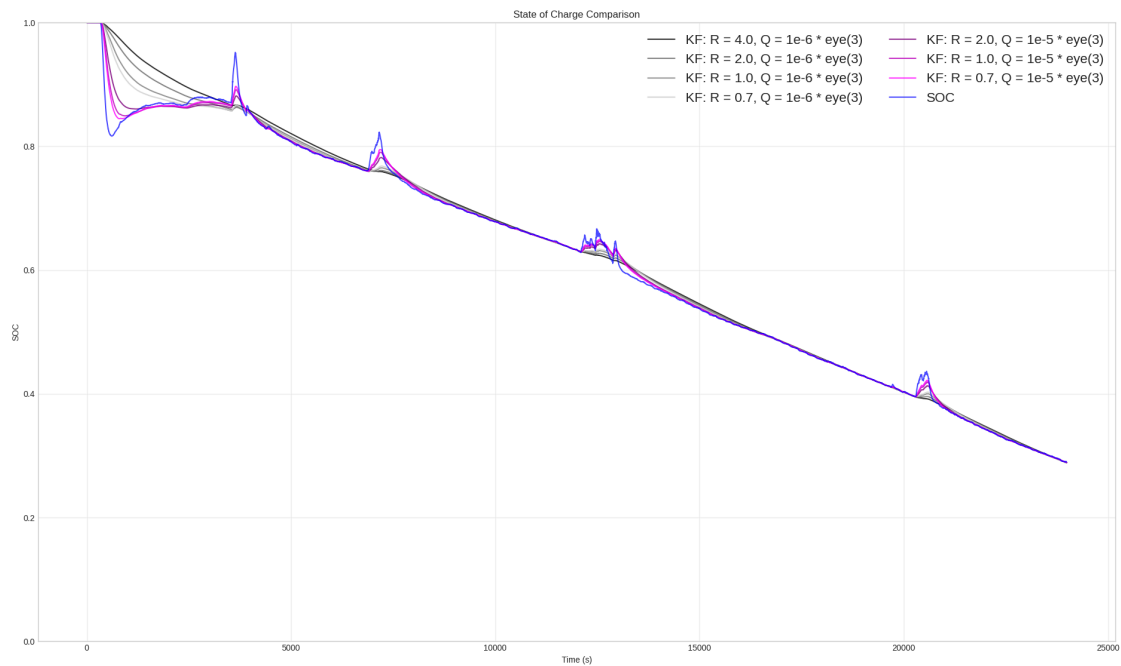


Abbildung 41: Kalman Filter tuning mit Initialwert = 1,0

Für eine bessere Untersuchung dient auf Abbildung 42 eine Detailansicht einer Fahr-Sequenz. Auch hier sind deutliche Verbesserungen ersichtlich.

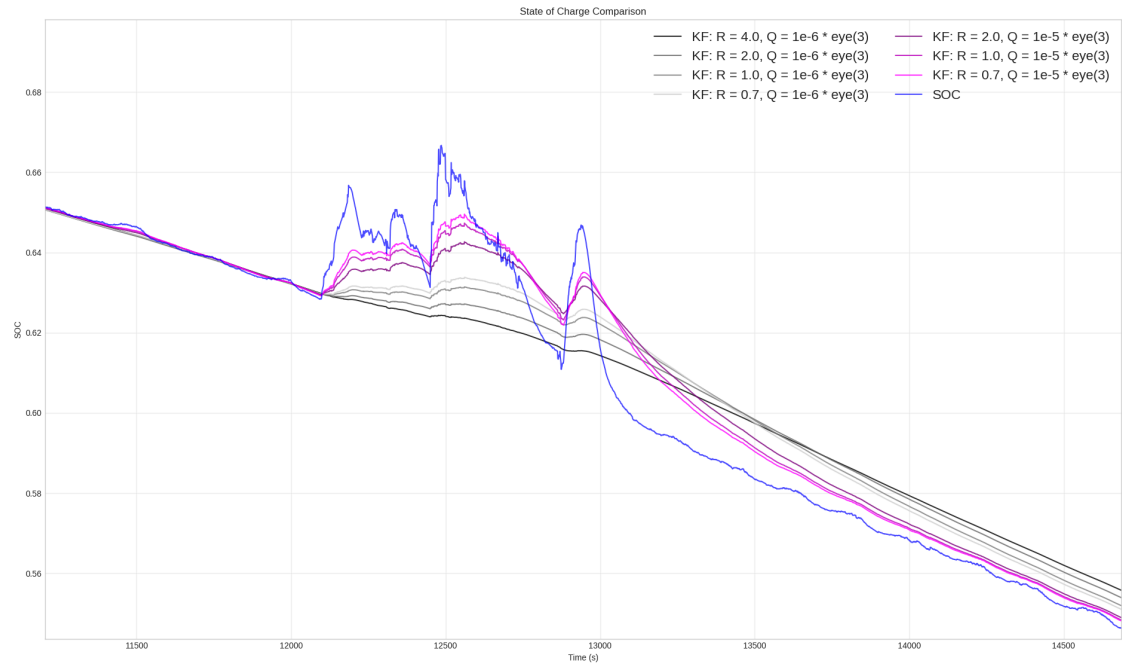


Abbildung 42: Detailaufnahme Kalman Filter tuning mit Initialwert = 1.0

Diese Glättung durch das stärkere Vertrauen in das Systemmodell hat zur Konsequenz, dass die Konvergenz zum wahren Wert länger dauert. Auf Abbildung 43 ist ersichtlich, dass Kennlinien mit starker Glättung nur sehr langsam zum wahren Wert konvergieren. Dazu wurden die Filter mit einem Initialwert von 0.5 definiert.

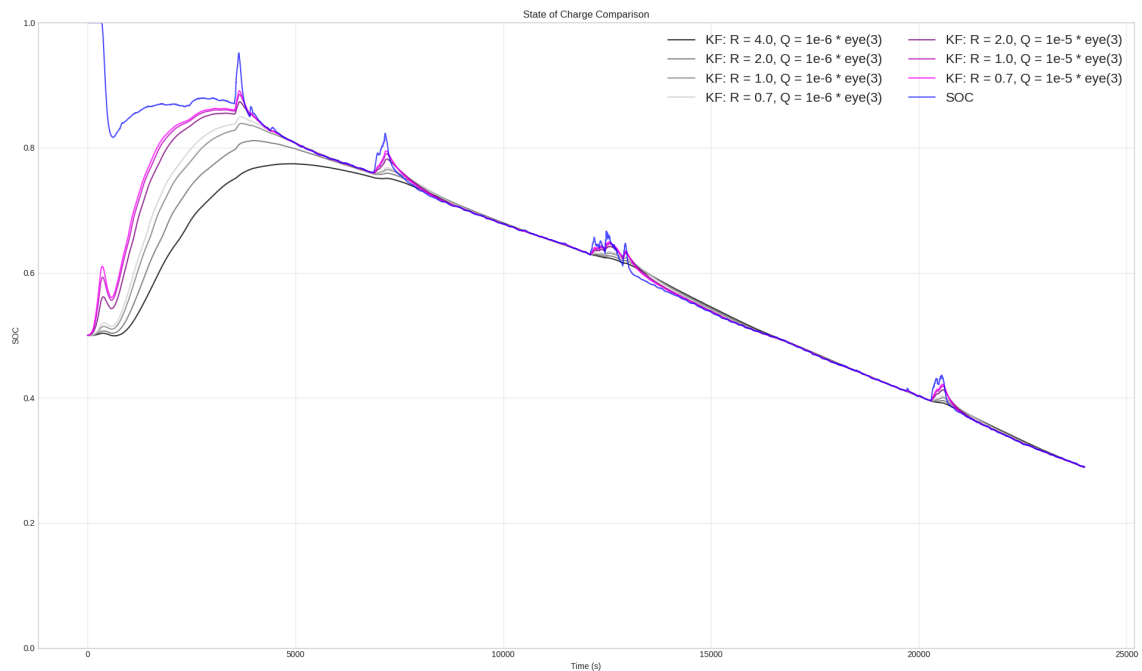


Abbildung 43: Kalman Filter tuning mit Initialwert = 0.5

Die Abbildung 44 zeigt die Detailansicht der unterschiedlichen Konvergenzen.

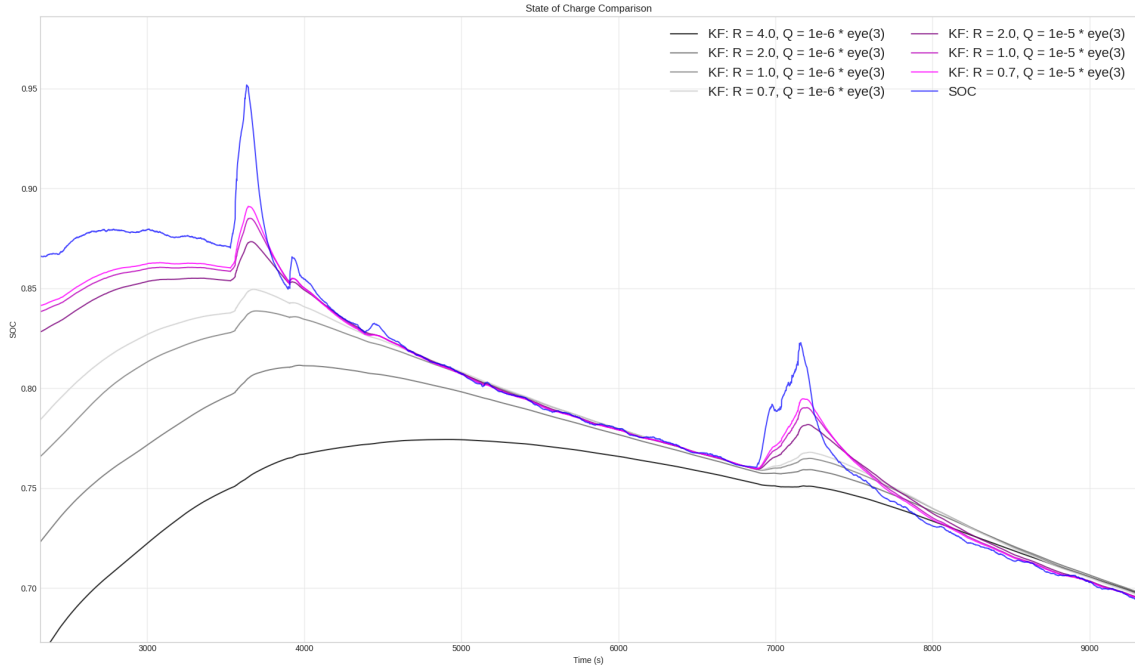


Abbildung 44: Detailaufnahme Kalman Filter tuning mit Initialwert = 0.5

Der Versuch mit unterschiedlichen Initialwerten soll aufzeigen, wie sich der Filter während des Betriebs bei Abweichungen verhält. Für das beim Roboter eingesetzte BMS sind die Werte $\mathbf{R} = 2.0$ und $\mathbf{Q} = 1e - 6 \cdot \text{eye}(3)$ gewählt worden.

Um nun die Initialisierung des Filters zu optimieren, lässt sich die Varianz des Startwert einstellen. Dazu dient der folgende Versuch auf Abbildung 45 mit unterschiedlichen Werten für die Kovarianzmatrix $\mathbf{P}$. Der blaue Verlauf zeigt dabei die Schätzung der ursprünglichen Einstellungen, während der rote Verlauf mit denselben Einstellungen jedoch geändertem Initialwert dargestellt ist. Alle anderen Schätzungen sind mit den zuvor gewählten Parametern für $\mathbf{R}$ und $\mathbf{Q}$ durchgeführt.

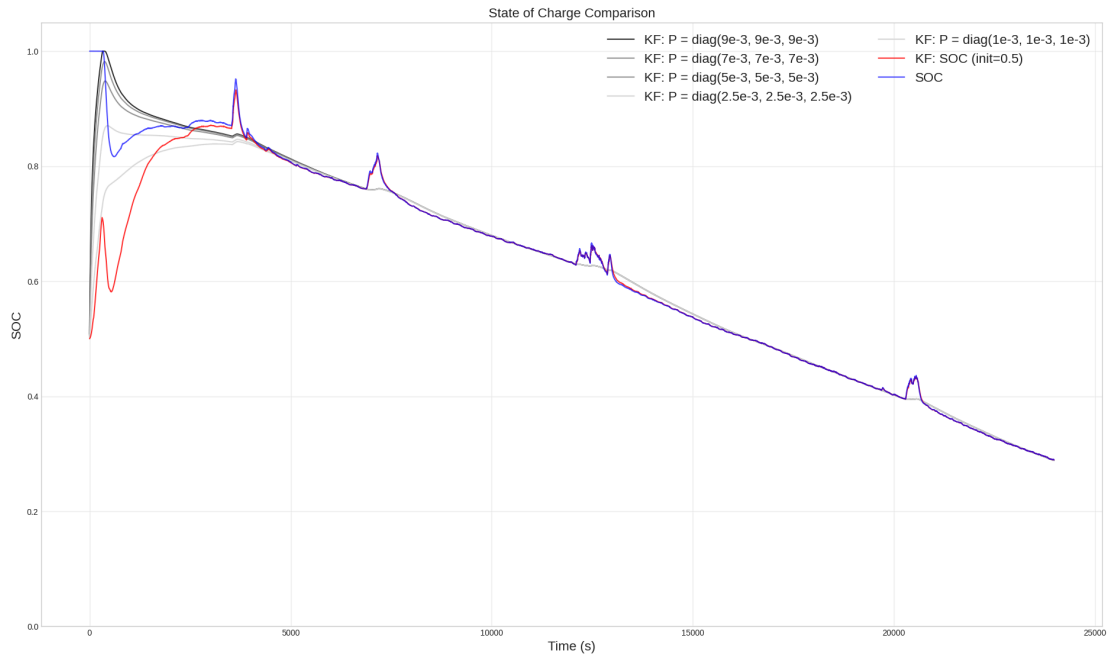


Abbildung 45: Detailaufnahme Kalman Filter tuning mit Initialwert = 0.5

Der Effekt der Kovarianzmatrix $\mathbf{P}$ ist deutlich zu sehen. Mit steigender Varianz nähert sich der Filter schneller dem wahren Wert an. Die folgende Abbildung 46 stellt eine Detailansicht der Startsequenz dar.

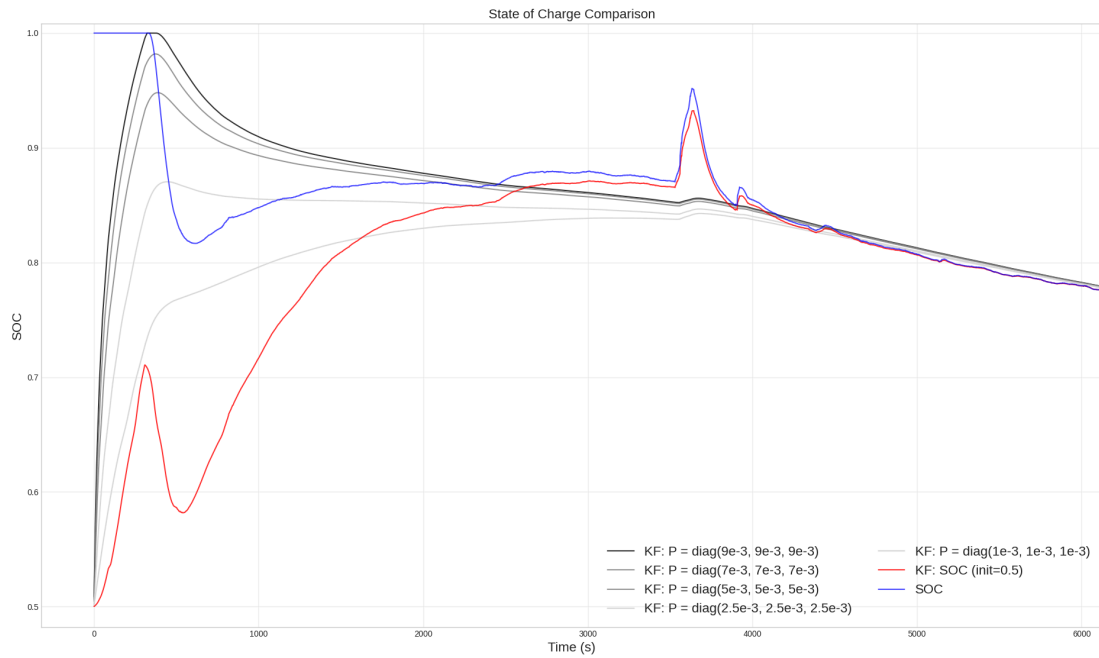


Abbildung 46: Detailaufnahme Kalman Filter tuning mit Initialwert = 0.5

Der vermutliche Idealwert liegt bei ungefähr einer Kovarianzmatrix von $\mathbf{P} = 7e-3 \cdot \text{eye}(3)$. Diese Kennlinie korrigiert rasch auf den wahren Wert, überschreitet jedoch nicht die Grenze von Eins.

4.1.3 Spannungsbasierte SOC-Schätzung

Die bisherige SOC-Schätzung basierte ausschliesslich auf Spannungsmessungen. Diese Methode erwies sich als unzuverlässig, da die Ladungsanzeige bei grösseren Belastungen deutlich einbrach. Die Daten des Langzeitversuchs auf Abbildung 40 dokumentieren diese Problematik eindeutig. Die Spannung schwankt bei jedem Belastungswechsel erheblich. Der kontinuierliche Grundverbrauch des Roboters verursacht zusätzlich einen permanenten Spannungsabfall an der Batterie. Diese beiden Faktoren verfälschen die SOC-Rückschlüsse aus Spannungsmessungen systematisch. Eine spannungsbasierte SOC-Schätzung kann durchaus präzise sein, da bei Blei-Säure Batterien zwischen SOC und OCV ein relativ linearer Zusammenhang besteht. Diese Methode erfordert jedoch zwingend einen offenen Schaltkreis ohne jegliche Belastung.

4.1.4 Abweichung bei der Ladung

Während der Ladung steigt der SOC-Wert zu schnell. Dies kann dadurch begründet werden, dass die Batterie nur während dem Entladevorgang charakterisiert wurde. Aufgrund der höheren Ladespannung wird die OCV der vollgeladenen Batterie zu früh erreicht.

4.2 State of Health

Die Validierung der SOH-Schätzung stellt eine besondere Herausforderung dar, da der Gesundheitszustand einer Batterie sich über Monate oder Jahre hinweg verändert. Um verlässliche Aussagen über die Genauigkeit der SOH-Schätzung treffen zu können, fehlen zurzeit Langzeitmessungen. Alternativ liessen sich Alterungsprozesse durch spezielle Laboreinrichtungen beschleunigen, was jedoch den Rahmen dieser Arbeit überschreiten würde. Die Literatur zeigt allerdings vielversprechende Ergebnisse für ähnliche Ansätze. Laut dem Paper *A review on the state of health estimation methods of lead-acid batteries*[10] konvergiert die SOH-Schätzung nach etwa 300 Betriebsstunden unabhängig vom gewählten Startwert. Dabei konnten sowohl SOC als auch SOH akkurat bestimmt werden. Diese Ergebnisse lassen erwarten, dass auch die in dieser Arbeit umgesetzte SOH-Schätzung bei längerer Betriebszeit gute Resultate erreichen kann.

Die folgende Abbildung 47 zeigt die Ergebnisse des bereits zuvor verwendeten Aufnahmen über eine längere Zeit.

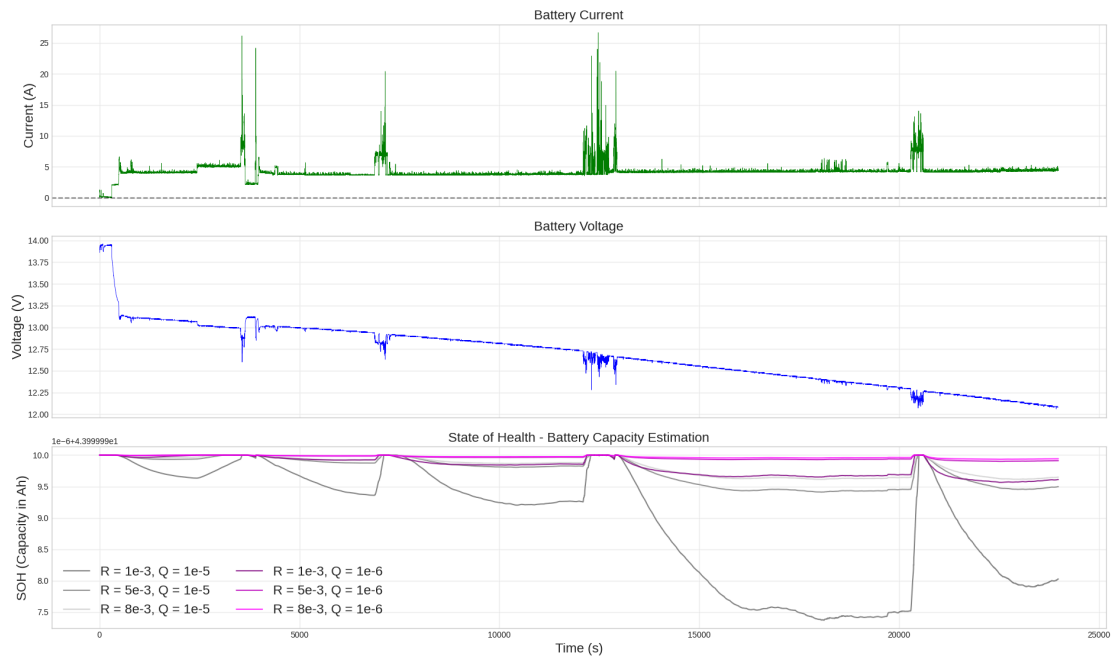


Abbildung 47: Parameter Vergleich SOH-Schätzung

Es ist ersichtlich, dass die Schätzung funktioniert und unterschiedliche Parameter das Resultat erwartungsgemäss beeinflussen. Jedoch ist eine Aufnahme über wenige Stunden nicht ausreichend, um effektiv Parameter zu optimieren.

4.3 Temperatur

Die Temperatur beeinflusst das Verhalten von Batterien erheblich. Diese Arbeit setzt einen Betrieb des Roboters in klimatisierten Räumen voraus. Für stabile Schätzungen bei variierenden Temperaturen sind Temperaturmessungen erforderlich. Zusätzlich müssten HPPC-Tests bei verschiedenen Temperaturbereichen durchgeführt werden, um entsprechende Lookup-Tabellen zu generieren.

5 Fazit

Das entwickelte Energie-Management-System erfüllt die gesetzten Ziele und bietet eine solide Grundlage für den zuverlässigen Betrieb des Roboters. Die Extended Kalman Filter-Ansätze zeigen deutliche Verbesserungen gegenüber spannungsbasierten Methoden, insbesondere bei wechselnder Belastung.

Die SOC-Schätzung erreicht durch die batteriespezifische Charakterisierung und das entwickelte Equivalent Circuit Model gute Ergebnisse. Der Filter konvergiert bei falschen Initialwerten innerhalb kurzer Zeit zum korrekten Wert und liefert auch bei unterschiedlichen Belastungen stabile Ergebnisse. Die Einstellparameter ermöglichen eine Anpassung zwischen Glättung und Konvergenzgeschwindigkeit je nach Bedarf.

Die Integration in das bestehende ROS2-System funktioniert zuverlässig und liefert kontinuierliche Daten. Die Energiemessungen ermöglichen Reichweitenberechnungen und bilden die Basis für intelligente Ladestrategien.

Einschränkungen bestehen bei der Validierung der SOH-Schätzung aufgrund der erforderlichen Langzeitmessungen. Zudem wurde die Temperaturabhängigkeit nicht berücksichtigt, was bei variierenden Umgebungsbedingungen zu Ungenauigkeiten führen kann. Um die SOC-Schätzung auch bei Ladung präzise abzubilden, braucht es weitere Analysen.

Die praktischen Tests zeigen die Funktionsfähigkeit des Battery Management System. Dies stellt eine deutliche Verbesserung für das Energiemanagement des Roboters dar und trägt dazu bei, dass der Roboter seine Aufgaben im Pflegeheim zuverlässiger erledigen kann.

6 Ausblick

Diese Arbeit implementierte die grundlegenden Funktionen eines BMS. Um die Leistung der beiden Filter zu optimieren, sollten in Zukunft weitere Daten erfasst werden. Langzeitmessungen könnten dazu dienen, die Filterparameter zu verbessern und die Robustheit des Systems zu steigern.

Das entwickelte BMS ermöglicht neue Funktionalitäten. Der Roboter kann Tasks unter Berücksichtigung eines intelligenten Energiemanagements abarbeiten. Verbrauchsschätzungen erlauben die Berechnung von Routen und Zeitplänen, während die Energieversorgung sichergestellt ist. Die Platzierung der Ladestation ist dabei auch ein Thema von Bedeutung. Eine optimierte Position kann zur Betriebseffizienz beitragen.

Eine weitere sinnvolle Systemerweiterung stellt die prädiktive Empfehlung für einen Batteriewechsel dar. Ein *Remaining Useful Life* (RUL) Algorithmus schätzt ab, wann die Batterieleistung unter die erforderliche Grenze fällt. Ergänzend könnte zudem ein *Fault Detection System* Ausfälle frühzeitig detektieren.

Abbildungsverzeichnis

Abbildung 1	3D-Modell WSR-Roboter	7
Abbildung 2	Konstruktion einer AGM Blei-Säure Batterie (Quelle: [2])	8
Abbildung 3	Auswirkung des DOD auf die Zyklenzahl (Quelle: [6])	9
Abbildung 4	Simulink Aufbau Kalman Filter vs. CC	13
Abbildung 5	Simulink Strom- und Spannungsverlauf	13
Abbildung 6	Simulink Aufbau Kalman Filter vs. CC	14
Abbildung 7	Schematischer Algorithmus Alpha-Beta Filter (Quelle: [14])	15
Abbildung 8	Diskrete Darstellung statisches System (Quelle: [14])	16
Abbildung 9	Gaussian-Glockenkurve (Quelle: [14])	17
Abbildung 10	Schematischer Algorithmus des Kalman Filter (Quelle: [14])	18
Abbildung 11	Kalman Filter mit $Q = 0.01$, $R = 0.4$	21
Abbildung 12	Kalman Filter mit $Q = 0.04$, $R = 0.4$	21
Abbildung 13	Kalman Filter mit $Q = 0.04$, $R = 0.2$	22
Abbildung 14	Kalman Filter mit $Q = 0.04$, $R = 0.8$	22
Abbildung 15	Berechnung Kalman Gain (Quelle: [14])	23
Abbildung 16	Gleichungen Kalman Filter Algorithmus (Quelle: [14])	23
Abbildung 17	Ballonmessung (Quelle: [14])	24
Abbildung 18	Nichtlineares System (Quelle: [14])	25
Abbildung 19	Linearisiertes System (Quelle: [14])	26
Abbildung 20	Equivalent Circuit Model (Quelle: [17])	30
Abbildung 21	Spannungsverlauf bei Stromsprung (Quelle:[17])	31
Abbildung 22	Test Aufbau	32
Abbildung 23	Vereinfachter Schaltkreis des Testaufbau	32
Abbildung 24	HPPC Rohdaten	33
Abbildung 25	Identifizierte Pulse mittels Matlab-Funktion	34
Abbildung 26	Optimierungsfunktion (curve fitting) von Matlab	35
Abbildung 27	Identifizierte Parameter	35
Abbildung 28	Vergleich Matlab-Simulation vs. EKF-Implementation	40
Abbildung 29	Detailvergleich Matlab-Simulation vs. EKF-Implementation	40
Abbildung 30	Equivalent Circuit Model (Quelle: [17])	42
Abbildung 31	Peacefair PZEM 017-3	43
Abbildung 32	Overview ROS2 Nodes	44
Abbildung 33	Energieverbrauch bei Roboter-Start	46
Abbildung 34	Energieverbrauch bei Fahrt	47
Abbildung 35		47
Abbildung 36	Batterie Ladekurve	50
Abbildung 37	Battery Monitor GUI	51

Abbildung 38 Funktionstest im Ruhezustand	52
Abbildung 39 Vergleich SOC-Schätzung bei realistischem Stromverbrauch	54
Abbildung 40 Langzeitaufnahme BMS-Daten	55
Abbildung 41 Kalman Filter tuning mit Initialwert = 1,0	56
Abbildung 42 Detailaufnahme Kalman Filter tuning mit Initialwert = 1.0	57
Abbildung 43 Kalman Filter tuning mit Initialwert = 0.5	57
Abbildung 44 Detailaufnahme Kalman Filter tuning mit Initialwert = 0.5	58
Abbildung 45 Detailaufnahme Kalman Filter tuning mit Initialwert = 0.5	59
Abbildung 46 Detailaufnahme Kalman Filter tuning mit Initialwert = 0.5	59
Abbildung 47 Parameter Vergleich SOH-Schätzung	61

Bibliographie

- [1] «Dingo Indoor Mobile Robot - Clearpath Robotics». Zugegriffen: 23. August 2025. [Online]. Verfügbar unter: <https://clearpathrobotics.com/dingo-indoor-mobile-robot/>
- [2] «How AGM Batteries Work – Sealed Lead-Acid Power Explained – DC Direct». Zugegriffen: 15. August 2025. [Online]. Verfügbar unter: <https://dcdirect.com.au/blogs/news/how-agm-batteries-work?srsId=AfmBOorX0HB8l0icY-e3cOGZIWKkRkbNf9WZc7PgKQOkkp12tHz31zFR>
- [3] «Bleiakkumulator – Wikipedia». Zugegriffen: 31. August 2025. [Online]. Verfügbar unter: <https://de.wikipedia.org/wiki/Bleiakkumulator>
- [4] «Was ist die C-Rate? Einfach & Ausführlich erklärt. | ChargeHere». Zugegriffen: 15. August 2025. [Online]. Verfügbar unter: <https://chargehere.de/glossar/c-rate/>
- [5] «Dingo Maintenance | Clearpath Robotics Documentation». Zugegriffen: 28. August 2025. [Online]. Verfügbar unter: https://docs.clearpathrobotics.com/docs_robots/legacy/ros1_robots/indoor_robots/dingo/maintenance_dingo/
- [6] «12V 22Ah Deep Cycle Sealed Lead Acid Battery with M5 Terminals - TLV12220M5DC». Zugegriffen: 4. August 2025. [Online]. Verfügbar unter: <https://www.upsbatterycenter.com/12v-22ah-deep-cycle-sealed-lead-acid-battery-with-m5-terminals>
- [7] M. Spendiff-Smith, «Complete Guide On How To Charge A Lead Acid Battery». Zugegriffen: 15. August 2025. [Online]. Verfügbar unter: <https://www.power-sonic.com/blog/how-to-charge-a-lead-acid-battery/>
- [8] «Peukert-Gleichung – Wikipedia». Zugegriffen: 15. August 2025. [Online]. Verfügbar unter: <https://de.wikipedia.org/wiki/Peukert-Gleichung>
- [9] «Remaining Useful Life - Battery Design». Zugegriffen: 4. August 2025. [Online]. Verfügbar unter: <https://www.batterydesign.net/remaining-useful-life/>
- [10] S. Jiang und Z. Song, «A review on the state of health estimation methods of lead-acid batteries», *Journal of Power Sources*, Bd. 517, S. 230710, Jan. 2022, doi: 10.1016/j.jpowsour.2021.230710.
- [11] O. Demirci, S. Taskin, E. Schaltz, und B. Acar Demirci, «Review of battery state estimation methods for electric vehicles-Part II: SOH estimation», *Journal of Energy Storage*, Bd. 96, S. 112703, Aug. 2024, doi: 10.1016/j.est.2024.112703.
- [12] O. Demirci, S. Taskin, E. Schaltz, und B. Acar Demirci, «Review of battery state estimation methods for electric vehicles - Part I: SOC estimation», *Journal of Energy Storage*, Bd. 87, S. 111435, Mai 2024, doi: 10.1016/j.est.2024.111435.

- [13] «ip8-roboter-energy-management/matlab at master · andriwild/ip8-roboter-energy-management». Zugegriffen: 21. August 2025. [Online]. Verfügbar unter: <https://github.com/andriwild/ip8-roboter-energy-management/tree/master/matlab>
- [14] A. Becker, *Kalman filter from the ground up*, Third edition. United States: Verlag nicht ermittelbar, 2024.
- [15] «Extended Kalman filter - Wikipedia». Zugegriffen: 14. August 2025. [Online]. Verfügbar unter: https://en.wikipedia.org/wiki/Extended_Kalman_filter
- [16] «ROS 2 Documentation — ROS 2 Documentation: Humble documentation». Zugegriffen: 8. August 2025. [Online]. Verfügbar unter: <https://docs.ros.org/en/humble/index.html>
- [17] «Explore Techniques to Estimate Battery State of Charge - MATLAB & Simulink». Zugegriffen: 27. Juli 2025. [Online]. Verfügbar unter: <https://ch.mathworks.com/help/simscape-battery/ug/estimate-battery-soc-using-kalman-filter-example.html>
- [18] R. Jackey, M. Saginaw, P. Sanghvi, J. Gazzarri, T. Huria, und M. Ceraolo, «Battery Model Parameter Estimation Using a Layered Technique: An Example Using a Lithium Iron Phosphate Cell», Apr. 2013, S. 2013–2011. doi: 10.4271/2013-01-1547.
- [19] «Kondensator (Elektrotechnik) – Wikipedia». Zugegriffen: 21. August 2025. [Online]. Verfügbar unter: https://de.wikipedia.org/wiki/Kondensator_%28Elektrotechnik%29?utm_source=chatgpt.com
- [20] S. Surya, J. Channegowda, S. D. Datar, A. S. Jha, und A. Victor, «Accurate Battery Modeling Based on Pulse Charging using MATLAB / Simulink», in *2020 IEEE International Conference on Power Electronics, Drives and Energy Systems (PEDES)*, Jaipur, India: IEEE, Dez. 2020, S. 1–3. doi: 10.1109/PEDES49360.2020.9379617.
- [21] «Estimate Equivalent Circuit Lithium-Ion Battery Data - MATLAB & Simulink». Zugegriffen: 29. August 2025. [Online]. Verfügbar unter: <https://ch.mathworks.com/help/autobkls/ug/estimate-equivalent-circuit-lithium-ion-battery-data.html>
- [22] «ip8-roboter-energy-management/hppc at master · andriwild/ip8-roboter-energy-management». Zugegriffen: 30. August 2025. [Online]. Verfügbar unter: <https://github.com/andriwild/ip8-roboter-energy-management/tree/master/hppc>
- [23] «SOH Estimator (Capacity-Based) - State of health estimator based on capacity fade - Simulink». Zugegriffen: 13. August 2025. [Online]. Verfügbar unter: <https://ch.mathworks.com/help/simscape-battery/ref/sohestimatorcapacitybased.html>
- [24] «Peacefair PZEM-003/017 300V Modbus RS485 Electric Voltmetro Ampere Kwh Wattmeter Smart Power Energie Meter DC-Peacefair PZEM-003/017 300V Modbus RS485 Electric Voltmetro Ampere Kwh Wattmeter Smart Power Energie Meter DC 价格-Peacefair PZEM-003/017 300V Modbus RS485 Electric Voltmetro Ampere Kwh Wattmeter Smart Power Energie Meter DC 批发-Peacefair PZEM-003/017 300V Modbus RS485 Electric

Voltmetro Ampere Kwh Wattmeter Smart Power Energie Meter DC 公司-Ningbo Peizheng Electronic Technology Co., Ltd». Zugriffen: 15. August 2025. [Online]. Verfügbar unter: <https://en.peacefair.cn/product/786.html>

- [25] «sensor_msgs/BatteryState Documentation». Zugriffen: 15. August 2025. [Online]. Verfügbar unter: https://docs.ros.org/en/noetic/api/sensor_msgs/html/msg/BatteryState.html
- [26] «ip8-roboter-energy-management/ros2 at master · andriwild/ip8-roboter-energy-management». Zugriffen: 29. August 2025. [Online]. Verfügbar unter: <https://github.com/andriwild/ip8-roboter-energy-management/tree/master/ros2>
- [27] «Volt, Ampere, Amperestunde, Watt und Wattstunde: Terminologie und Leitfaden». Zugriffen: 23. August 2025. [Online]. Verfügbar unter: <https://www.rebel-cell.com/de/grundlagenwissen/akku-terminologie/>

Hiermit bestätige ich, dass die eingereichte Projektarbeit mit dem Titel «Energie-Management in mobilen Robotern» das Resultat meiner persönlichen Erarbeitung der Themen ist. Ich habe keine anderen als die angegebenen Quellen benutzt.

Dagmersellen, 31.08.2025

Andri Wild